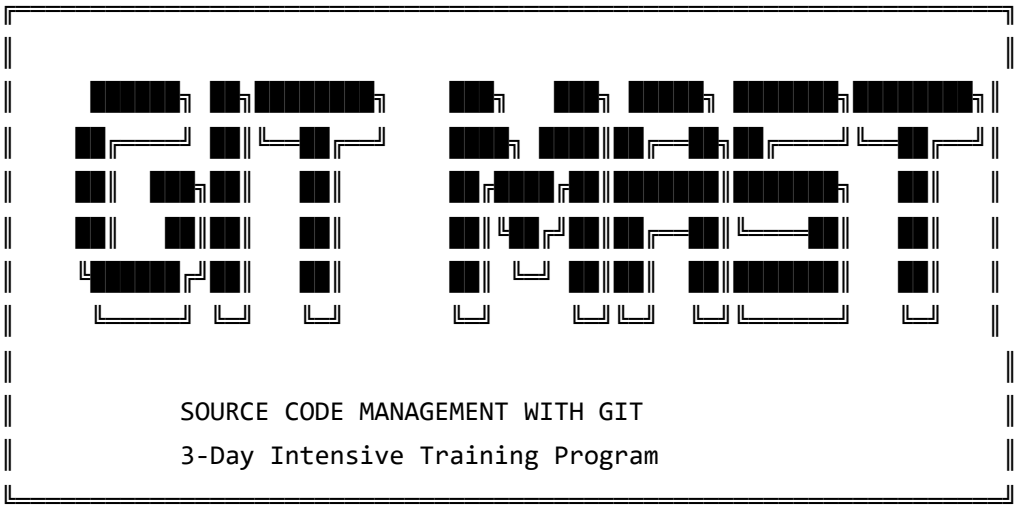
Source Code Management with Git

A Comprehensive 3-Day Training Guide

From Absolute Beginner to Expert Mastery



📋 Program Overview

Day	Focus	Sections
Day 1	Git Foundations & Core Concepts	3.0 – 3.2
Day 2	Branching, Merging & Advanced Techniques	4.0 – 4.1
Day 3	Collaboration, Remote Workflows & Code Review	4.2 – 4.3

Target Audience: Complete beginners with no prior coding or programming experience
Format: Hands-on, example-driven, visually rich
Goal: Take you from zero to professional-level Git proficiency

Table of Contents

Section 3.0

Source Code Management with Git

🎯 Section Learning Objectives

By the end of this section, you will be able to:

- Explain what source code management means and why it matters
- Understand the fundamental difference between files with and without version control
- Describe how Git fits into the modern software development ecosystem
- Articulate the business value of using a version control system

Introduction: What is Source Code Management?

Imagine you're writing a novel. You save the file as `novel.docx`. Then you make changes and call it `novel_v2.docx`. Then you share it with a friend who makes edits: `novel_v2_JOHN_EDITS.docx`. Two weeks later you have `novel_FINAL.docx`, `novel_FINAL_REAL.docx`, and `novel_FINAL_PLEASE_USE_THIS_ONE.docx`. Sound familiar?

Now imagine that chaos applied to software code — but with a team of 50 people, thousands of files, and millions of lines of code. That's the problem **Source Code Management (SCM)** solves.

Source Code Management (also called **Version Control**) is a system that:

1. Tracks every change ever made to your files
2. Records *who* made each change and *when*
3. Lets you travel back in time to any previous version
4. Allows multiple people to work on the same files simultaneously without conflict

Git is the world's most popular SCM tool. Created by Linus Torvalds (the creator of Linux) in 2005, Git is now used by over 90% of professional software developers worldwide.

```
WITHOUT VERSION CONTROL          WITH GIT
═══════════════════════          ═══════════════════════════════════

novel.docx                       ┌─ Commit: "Initial draft"
novel_v2.docx                    │    Author: You
novel_v2_JOHN.docx               │    Date: Jan 1
novel_FINAL.docx                 ├─ Commit: "John's edits added"
novel_FINAL2.docx                │    Author: John
novel_FINAL_REAL.docx            │    Date: Jan 3
novel_USE_THIS.docx              ├─ Commit: "Fixed typos in ch. 2"
                                 │    Author: You
😱 CHAOS!                        │    Date: Jan 5
                                 └─ Commit: "Final review complete"
                                      Author: Editor
                                      Date: Jan 7

                                 ✅ ORGANIZED, TRACKABLE, REVERSIBLE!
```

Section 3.1

Version Control with Git

Git Introduction

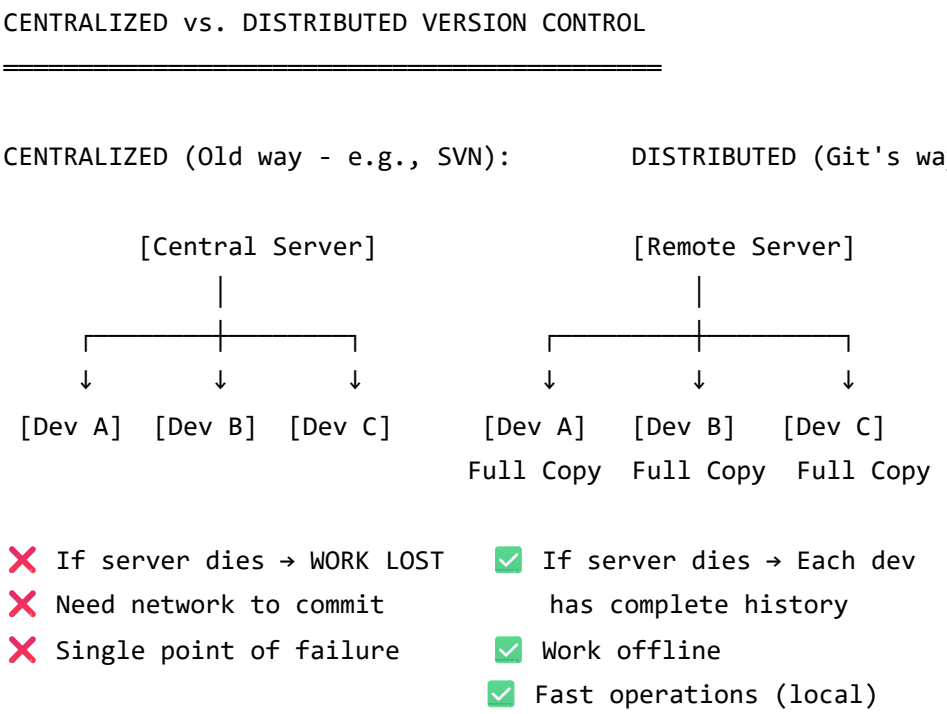
🎯 Learning Objectives

- Understand what Git is and how it differs from other version control systems
- Learn the core terminology: repository, commit, branch, merge
- Understand Git's distributed architecture
- Recognize how Git fits into the developer's daily workflow

What Exactly is Git?

Git is a **Distributed Version Control System (DVCS)**. Let's break that down:

- **Distributed** → Every person has a complete copy of the entire project history on their own computer
- **Version Control** → It tracks and manages changes to files over time
- **System** → It's a tool/software you install and use via commands



Key Git Concepts at a Glance

Term	What It Means	Real-World Analogy
Repository (Repo)	The project folder that Git tracks	A filing cabinet for your project
Commit	A saved snapshot of your work	Pressing "Save" with a description
Branch	A parallel version of your project	A separate notebook for experiments
Merge	Combining two branches together	Merging two notebooks into one
Clone	Downloading a copy of a repository	Photocopying a complete filing cabinet
Push	Uploading your changes to a server	Sending your work to the office
Pull	Downloading others' changes	Receiving updates from the office
Fork	Your own copy of someone else's project	Making your own copy of a recipe

Pro Tip 💡

> Git and GitHub are **not** the same thing! **Git** is the tool you install on your computer. **GitHub** (or GitLab, Bitbucket) is a website where you can store and share your Git repositories online. Think of Git as Microsoft Word, and GitHub as Google Drive — one is the tool, the other is storage.

Why Version Control?

🎯 Learning Objectives

- Understand the 5 core problems that version control solves
- Recognize real-world scenarios where Git saves the day
- Appreciate the collaborative benefits of version control
- Understand the cost of NOT using version control

The 5 Problems Version Control Solves

Problem 1: The "I Broke Everything" Problem

```
SCENARIO:
You're editing your company website. You make 20 changes.
Suddenly, nothing works. You don't know which change broke it.

WITHOUT GIT:                    WITH GIT:
❌ Manually undo 20 changes     ✅ git revert HEAD
❌ Hope you remember what       ✅ Instantly back to working state
   you changed                  ✅ See exactly which commit broke it
❌ Possibly lose hours of work
```

Problem 2: The "Two People, One File" Problem

```
SCENARIO:
Alice and Bob both edit the same file at the same time.

WITHOUT GIT:                    WITH GIT:
❌ Last save wins               ✅ Both changes preserved
❌ Alice's work deleted         ✅ Git identifies conflicts
❌ No warning given             ✅ Team resolves conflicts manually
                                ✅ Nothing is silently lost
```

Problem 3: The "What Changed?" Problem

```
SCENARIO:
A bug appeared last Tuesday. What code changed that day?

WITHOUT GIT:                    WITH GIT:
❌ No record of changes          ✅ git log shows all commits
❌ Ask everyone what they did    ✅ git diff shows exact changes
❌ Read through thousands        ✅ Find the bug in minutes
   of lines manually
```

Problem 4: The "Experimental Features" Problem

```
SCENARIO:
You want to try a risky new feature without breaking the live app.

WITHOUT GIT:                    WITH GIT:
❌ Copy the entire project       ✅ git branch new-feature
   folder (messy!)              ✅ Experiment safely
❌ Risk breaking production      ✅ Merge when ready
❌ Hard to sync changes later    ✅ Delete if experiment fails
```

Problem 5: The "Who Did This?" Problem

```
SCENARIO:
A critical bug was introduced. Who wrote this code and why?

WITHOUT GIT:                    WITH GIT:
❌ No way to know                ✅ git blame shows author of each line
❌ Blame game begins             ✅ See the commit message explaining why
❌ No context for decisions      ✅ Discuss with the right person
```

Real-World Impact

Companies that use Git effectively:

- **Deploy code faster** (10x-100x more frequently than companies without version control)
- **Recover from mistakes** in minutes instead of hours or days
- **Onboard new team members** quickly (they can see the entire project history)
- **Work globally** with teams across different time zones seamlessly

Git Installation and Configuration

🎯 Learning Objectives

- Install Git on your operating system
- Configure your identity (name and email)
- Understand why configuration matters
- Set up a comfortable Git environment

Installing Git

On Windows:

1. Visit git-scm.com
2. Download "Git for Windows"
3. Run the installer (accept all defaults for beginners)
4. Open "Git Bash" from your Start menu

On Mac:

```
# Option 1: Install Homebrew first (recommended), then Git
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)'
brew install git

# Option 2: Simply type git in terminal — macOS will prompt you to install
git --version
```

On Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install git
```

Verify Installation:

```
git --version
# Expected output: git version 2.43.0 (or similar)
```

Configuring Your Identity

Before using Git, you must tell it who you are. This information is attached to every commit you make.

```
# Set your name (use your real name — this appears in commit history)
git config --global user.name "Jane Smith"

# Set your email (use the same email as your GitHub account)
git config --global user.email "jane.smith@example.com"

# Set your preferred text editor for writing commit messages
git config --global core.editor "nano"    # Beginner-friendly
# OR
git config --global core.editor "code --wait"  # VS Code (recommended)

# Set the default branch name to "main" (modern standard)
git config --global init.defaultBranch main

# Verify your configuration
git config --list
```

Example output of `git config --list`:

```
user.name=Jane Smith
user.email=jane.smith@example.com
core.editor=code --wait
init.defaultbranch=main
```

Understanding the `--global` Flag

```
Git Configuration Levels:
═══════════════════════════

  --system   → Applies to ALL users on this computer
               File: /etc/gitconfig

  --global   → Applies to YOUR USER ACCOUNT (most common)
               File: ~/.gitconfig

  (no flag)  → Applies ONLY to the current project
               File: .git/config

  Priority: Project > Global > System
  (more specific settings override broader ones)
```

Helpful Configuration Extras

```
# Make Git output colorful and easier to read
git config --global color.ui auto

# Set up a useful alias (shortcut)
git config --global alias.st status     # Now 'git st' works like 'git status'
git config --global alias.lg "log --oneline --graph --decorate"  # Pretty log

# Configure line ending handling (important for cross-platform teams)
# On Windows:
git config --global core.autocrlf true
# On Mac/Linux:
git config --global core.autocrlf input
```

Common Mistakes ⚠️

- **Typos in your email** — This email links your commits to your GitHub account. A typo means your contributions won't be attributed correctly.
- **Skipping configuration** — Git will still work, but commit history will show generic/wrong author info.
- **Forgetting** `--global` — Without it, config only applies to the current project.

Git Basics

The Git Mental Model

Before learning commands, you need to understand Git's internal model. Git thinks about your files in terms of **three areas**:

```
THE THREE AREAS OF GIT
═══════════════════════════════════════════════════════════════

┌─────────────┐    git add    ┌─────────────┐  git commit  ┌─────────────┐
│             │ ────────────► │             │ ───────────► │             │
│  WORKING    │               │   STAGING   │              │   COMMIT    │
│  DIRECTORY  │               │    AREA     │              │   HISTORY   │
│             │ ◄──────────── │  (Index)    │              │ (Repository)│
│  Your files │ git checkout  │             │              │  Database)  │
│  as you see │               │  Files      │              │             │
│  them       │               │  staged for │              │  Permanent  │
│             │               │  next commit│              │  snapshots  │
└─────────────┘               └─────────────┘              └─────────────┘

REAL-WORLD ANALOGY:

Working Directory = Your desk (messy, work in progress)
Staging Area      = A box you're packing to ship
Commit History    = Packages that have been shipped and received

You carefully choose what goes in each package (commit)
before shipping it. Shipping is permanent!
```

This three-area model is the most important concept to grasp. Many Git confusions stem from not understanding this.

Initializing a Repository

🎯 Learning Objectives

- Create a new Git repository from scratch
- Understand what the `.git` folder is and why you shouldn't touch it
- Know the difference between `git init` and cloning an existing repo
- Successfully initialize your first project

The `git init` Command

`git init` tells Git to start tracking a folder. It creates a hidden `.git` directory that contains all of Git's tracking information.

```
# Example 1: Create a new project and initialize Git
mkdir my-first-project           # Create a new folder
cd my-first-project              # Navigate into it
git init                         # Initialize Git

# Expected output:
# Initialized empty Git repository in /home/jane/my-first-project/.git/


# Example 2: Initialize Git in an existing project folder
cd existing-project              # Navigate to your existing folder
git init                         # Add Git tracking to it

# Git doesn't delete any of your existing files!
# It just adds the .git folder


# Example 3: Verify the .git folder was created
ls -la                           # -la shows hidden files

# You should see:
# drwxr-xr-x  .git/             ← This is Git's "brain"
# (plus any other files in your project)
```

What's Inside the `.git` Folder?

```
.git/
├── HEAD          ← Points to your current branch
├── config        ← Project-specific Git settings
├── objects/      ← Where Git stores all your file snapshots
│   ├── info/
│   └── pack/
├── refs/         ← Stores branch and tag pointers
│   ├── heads/    ← Local branches
│   └── tags/     ← Tags (named releases)
├── hooks/        ← Scripts that run automatically (advanced)
└── index         ← The staging area
```

⚠️ NEVER manually edit files inside .git/
Git manages this entirely on its own.
Touching it can corrupt your repository!

Practical Example: Starting a Real Project

```
# Let's set up a sample website project

mkdir my-website
cd my-website
git init

# Create some initial files
echo "<!DOCTYPE html><html><body><h1>Hello World</h1></body></html>" > index.html
echo "body { font-family: Arial; }" > style.css
echo "# My Website" > README.md

# Check Git's current status
git status

# Expected output:
# On branch main
# No commits yet
#
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#         index.html
#         style.css
#         README.md
```

Hands-on Exercise 1: Your First Repository

Task: Create a portfolio project repository

```
# Step 1: Create the project
mkdir my-portfolio
cd my-portfolio
git init

# Step 2: Create some files
echo "# My Portfolio" > README.md
echo "My name is [Your Name]" >> README.md
mkdir projects
touch projects/project1.txt

# Step 3: Verify Git is tracking the folder
git status
# Should show: README.md and projects/ as untracked

# Step 4: Confirm the .git folder exists
ls -la
# Should see .git in the list
```

Expected result: A Git-initialized project with untracked files ready to be staged.

Common Mistakes ⚠️

- **Running `git init` inside an existing Git repo** — This creates a "nested repository" which causes confusion. Always check with `git status` first.
- **Deleting the `.git` folder** — This destroys ALL version history. The folder is Git.
- **Initializing Git in your home directory** — Only initialize Git in your project folders, not in `/home/username/` or `C:\Users\YourName\`.

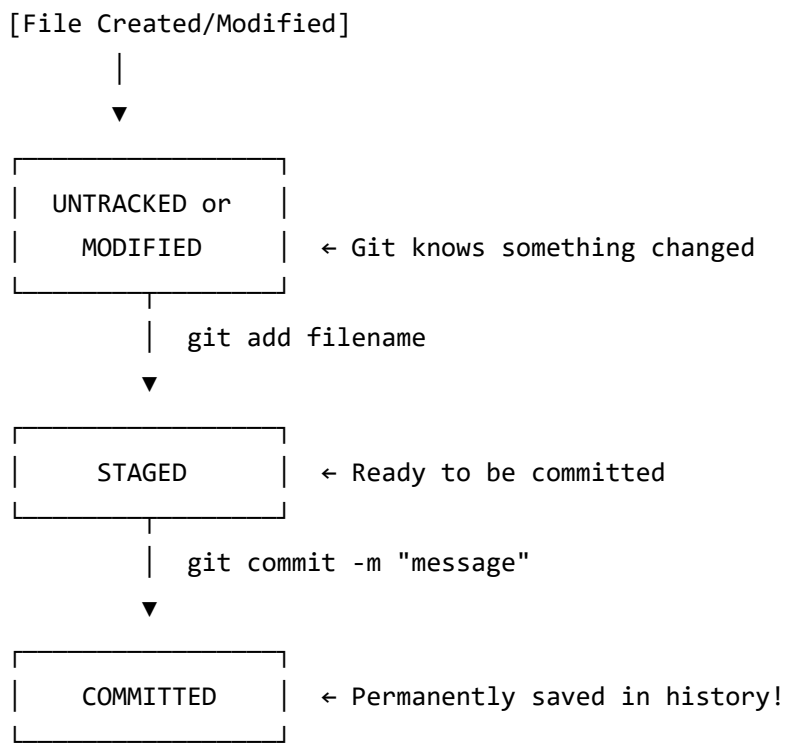
Staging and Committing Changes

🎯 Learning Objectives

- Use `git add` to stage files for a commit
- Use `git commit` to save a permanent snapshot
- Write meaningful commit messages
- Understand the staging-to-commit workflow completely

The Workflow: Add → Commit

```
HOW STAGING AND COMMITTING WORKS
═══════════════════════════════════════════════════════════════

Step 1: You edit files in your Working Directory
         (Git sees them as "modified" or "untracked")

Step 2: You stage the files you WANT to include in the commit
         git add filename.txt   ← Stage a specific file
         git add .              ← Stage ALL changed files

Step 3: You commit — creating a permanent snapshot
         git commit -m "Your description here"

═══════════════════════════════════════════════════════════════
```



The `git add` Command

```
# Add a single file
git add index.html

# Add multiple specific files
git add index.html style.css

# Add all files in current directory (and subdirectories)
git add .

# Add all files with a specific extension
git add *.js

# Add a specific folder
git add src/

# Add parts of a file interactively (advanced — choose specific lines)
git add -p filename.txt

# Check what's staged vs. not staged
git status
```

The `git commit` Command

```
# Commit with a short message (most common)
git commit -m "Add homepage HTML structure"

# Commit with a detailed message (opens your text editor)
git commit
# This opens your editor. Write a subject line, blank line, then details.

# Stage AND commit all tracked files in one command (skip git add)
git commit -am "Fix typo in homepage"
# ⚠️ -a only works for files Git is ALREADY tracking (not new files)

# Amend the last commit (fix a typo in your message or add forgotten file)
git commit --amend -m "Corrected commit message"
# ⚠️ Only amend commits that haven't been pushed to a shared server!
```

Writing Good Commit Messages

A commit message is a note to your future self and your teammates.

ANATOMY OF A GREAT COMMIT MESSAGE

Subject line (50 chars or less):

```
┌──────────────────────────────────────────────────┐
│ Add user authentication with JWT tokens          │
└──────────────────────────────────────────────────┘

↑ Use imperative mood: "Add" not "Added" or "Adding"
↑ Capitalize first letter
↑ No period at the end
↑ Describe WHAT changed, not HOW
```

Blank line separating subject from body

Body (72 chars per line, optional but valuable for complex changes):

```
┌──────────────────────────────────────────────────┐
│ Implement JWT-based authentication to replace    │
│ session cookies. This improves stateless API     │
│ design and allows for easier horizontal scaling. │
│                                                  │
│ - Add JWT middleware to Express server           │
│ - Update login endpoint to return tokens         │
│ - Add token refresh logic                        │
│ - Update tests for new auth flow                 │
└──────────────────────────────────────────────────┘
```

❌ BAD COMMIT MESSAGES:

"fix bug"
"updates"
"asdfgh"
"final version"
"AAAAAA WHY ISNT THIS WORKING"

✅ GOOD COMMIT MESSAGES:

"Fix null pointer exception in user login"
"Add dark mode toggle to settings page"
"Update README with installation instructions"
"Refactor database queries for 40% performance gain"

Complete Practical Walkthrough

```
# === Scenario: Building a simple website ===

# 1. Check current status
git status
# Output: nothing to commit, working tree clean  (or shows changes)

# 2. Create and edit a file
echo "<h1>Welcome to My Site</h1>" > index.html

# 3. Check status again
git status
# Output:
# Untracked files:
#         index.html

# 4. Stage the file
git add index.html

# 5. Check status after staging
git status
# Output:
# Changes to be committed:
#         new file:   index.html

# 6. Make the commit
git commit -m "Add initial homepage HTML"
# Output:
# [main (root-commit) a3f91b2] Add initial homepage HTML
#  1 file changed, 1 insertion(+)
#  create mode 100644 index.html

# 7. Add more files and commit together
echo "body { margin: 0; }" > style.css
echo "console.log('Hello!');" > app.js

git add style.css app.js
git commit -m "Add CSS styles and JavaScript entry point"

# 8. Modify an existing file
echo "<p>More content here</p>" >> index.html

git status
# Output:
# Changes not staged for commit:
```

```
#         modified:   index.html

git add index.html
git commit -m "Add introductory paragraph to homepage"

# 9. View the history of your commits
git log --oneline
# Output:
# c8d2f14 Add introductory paragraph to homepage
# b7e1a93 Add CSS styles and JavaScript entry point
# a3f91b2 Add initial homepage HTML
```

The `.gitignore` File — Excluding Files from Git

Some files should NEVER be committed: passwords, system files, generated files.

```
# Create a .gitignore file in your project root
nano .gitignore
```

```
# .gitignore examples
# Lines starting with # are comments

# Operating system files
.DS_Store          # Mac system file
Thumbs.db          # Windows thumbnail file

# Dependency folders (install with npm install, not stored in Git)
node_modules/

# Environment files (contains passwords/API keys — NEVER commit these!)
.env
.env.local

# Build output (generated automatically)
dist/
build/
*.log

# IDE/editor settings
.vscode/
.idea/

# Compiled files
*.pyc
*.class


# After creating .gitignore, add and commit it
git add .gitignore
git commit -m "Add .gitignore to exclude system and dependency files"
```

Hands-on Exercise 2: Your First Commits

Scenario: You're building a recipe website.

```
# Setup
mkdir recipe-website
cd recipe-website
git init

# Task 1: Create project structure
mkdir css js images
echo "# Recipe Website" > README.md
echo "Welcome to my recipe collection!" >> README.md

# Task 2: Create your first commit
git add README.md
git commit -m "Initialize project with README"

# Task 3: Add HTML file
cat > index.html << 'EOF'
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Recipe Website</title>
</head>
<body>
    <h1>My Favorite Recipes</h1>
    <ul>
        <li>Pasta Carbonara</li>
        <li>Chocolate Cake</li>
    </ul>
</body>
</html>
EOF

git add index.html
git commit -m "Add homepage with recipe list"

# Task 4: Add styles
echo "body { font-family: Georgia, serif; background: #f9f5f0; }" > css/style.css
git add css/style.css
git commit -m "Add base styles for warm recipe aesthetic"

# Task 5: Create .gitignore
echo ".DS_Store" > .gitignore
echo "*.log" >> .gitignore
git add .gitignore
git commit -m "Add gitignore for system files"
```

```
# View your history
git log --oneline
```

Best Practices for Staging & Committing

1. **Commit early, commit often** — Small commits are easier to understand and revert
2. **One logical change per commit** — Don't mix "fix login bug" with "update homepage design"
3. **Always check `git status` before committing** — Know exactly what you're committing
4. **Never commit sensitive data** — Passwords, API keys, database credentials
5. **Write commit messages in present tense** — "Add feature" not "Added feature"

Quick Reference — Core Commands

```
git add <file>          Stage a specific file
git add .               Stage all changes
git add -p              Stage changes interactively (line by line)
git commit -m "msg"     Commit with message
git commit -am "msg"    Stage (tracked files) + commit in one step
git status              See what's staged, modified, untracked
git diff                See unstaged changes
git diff --staged       See staged changes (before committing)
```

Section 3.2

Git Operations and Workflow

Managing Changes

🎯 Learning Objectives

- Use `git status` and `git diff` to understand the state of your repository
- Compare file versions before and after changes
- Understand what information Git tracks for each change
- Navigate common change-management scenarios

Understanding `git status`

`git status` is your most-used command. Run it constantly!

```
git status

# Possible outputs explained:

# === CLEAN WORKING TREE ===
# On branch main
# nothing to commit, working tree clean
# → You have no uncommitted changes

# === UNTRACKED FILES ===
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#         newfile.txt
# → Git sees this file but isn't tracking it yet

# === MODIFIED FILES ===
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#         modified:   index.html
# → File exists and is tracked, but has unsaved (unstaged) changes

# === STAGED FILES ===
# Changes to be committed:
#   (use "git restore --staged <file>..." to unstage)
#         new file:   newfile.txt
#         modified:   index.html
# → These changes will be included in your next commit
```

Understanding `git diff`

`git diff` shows you the exact content that changed.

```
# See changes NOT YET staged
git diff

# See changes ALREADY staged (will go into next commit)
git diff --staged

# Compare two specific commits
git diff abc1234 def5678

# Compare a file between two commits
git diff abc1234 def5678 -- index.html

# Example output of git diff:
# diff --git a/index.html b/index.html
# index a1b2c3d..e4f5g6h 100644
# --- a/index.html          ← "Before" version
# +++ b/index.html          ← "After" version
# @@ -1,5 +1,6 @@
#  <!DOCTYPE html>
#  <html>
#  <body>
# -    <h1>Hello</h1>       ← RED = deleted/removed line
# +    <h1>Hello World</h1> ← GREEN = added/new line
# +    <p>Welcome!</p>      ← GREEN = added/new line
#  </body>
#  </html>
```

Viewing Commit History

🎯 Learning Objectives

- Use `git log` to view project history
- Filter and format log output for readability
- Find specific commits using search options
- Use `git show` to inspect individual commits

The `git log` Command

```
# Basic log — shows full commit details
git log

# Each commit shows:
# commit a3f91b2c4d5e6f7g8h9i0j1k2l3m4n5o6p7q  ← Unique commit ID (SHA hash)
# Author: Jane Smith <jane@example.com>           ← Who made the commit
# Date:   Mon Jan 15 09:23:41 2024 +0000          ← When
#
#     Add homepage HTML structure                 ← Commit message
#
# (press Q to exit the log)

# Compact one-line format (most useful day-to-day)
git log --oneline
# Output:
# c8d2f14 Add introductory paragraph to homepage
# b7e1a93 Add CSS styles and JavaScript entry point
# a3f91b2 Add initial homepage HTML

# Visual branch graph (essential when working with branches)
git log --oneline --graph --decorate
# Output:
# * c8d2f14 (HEAD -> main) Add introductory paragraph
# * b7e1a93 Add CSS styles and JavaScript
# * a3f91b2 Add initial homepage HTML

# Show last N commits
git log -5                    # Last 5 commits
git log --oneline -10         # Last 10 commits, one line each

# Search by author
git log --author="Jane"

# Search by date range
git log --after="2024-01-01" --before="2024-02-01"

# Search by commit message keyword
git log --grep="homepage"

# Show commits that changed a specific file
git log -- index.html

# Show what files were changed in each commit
git log --stat
```

```
# Show the actual changes (diff) for each commit
git log -p
```

Reading the Commit Graph

```
git log --oneline --graph --decorate --all

* a1b2c3d (HEAD -> main) Fix navigation bug        ← Current position (HEAD)
* d4e5f6g Add contact page
* g7h8i9j Add about page
* j1k2l3m Initial commit                           ← Oldest commit (bottom)

With branches:
*   m1n2o3p (HEAD -> main) Merge feature/login      ← Merge commit
|\
| * p4q5r6s (feature/login) Add password validation
| * s7t8u9v Add login form HTML
|/
* v1w2x3y Add homepage
```

Using `git show` to Inspect a Commit

```
# Show details of a specific commit
git show a3f91b2

# Shows:
# - Commit metadata (author, date, message)
# - The complete diff of what changed

# Show just the files changed in a commit
git show --stat a3f91b2

# Show the most recent commit
git show HEAD

# Show the commit 2 before HEAD
git show HEAD~2
```

Hands-on Exercise 3: Exploring History

```
# Use the recipe-website project from Exercise 2

# Task 1: View the full history
git log

# Task 2: View compact history
git log --oneline

# Task 3: See what changed in a specific commit
# (replace abc1234 with your actual commit hash)
git show abc1234

# Task 4: See history with file details
git log --stat

# Task 5: Find commits mentioning "styles"
git log --grep="styles"

# Challenge: View only the last 2 commits in oneline format
git log --oneline -2
```

Undoing Changes

🎯 Learning Objectives

- Understand the difference between `git revert` and `git reset`
- Know when to use each undo technique safely
- Restore individual files to previous states
- Understand the risks of rewriting history

The Undo Decision Tree

```
CHOOSING THE RIGHT UNDO COMMAND
═══════════════════════════════════════════════════════════

  Did you commit yet?
  │
  ├─ NO (changes are unstaged or staged)
  │   │
  │   ├─ Unstaged: git restore <file>        (discard working changes)
  │   └─ Staged:   git restore --staged <file>  (unstage, keep changes)
  │
  └─ YES (already committed)
      │
      ├─ Was it pushed to a shared server?
      │   │
      │   ├─ NO  → git reset (rewrite history locally)
      │   └─ YES → git revert (safe undo — creates new commit)
```

`git restore` — Undoing Before Committing

```
# SCENARIO 1: You edited a file but want to throw away those changes
# (goes back to the last committed version)

git restore index.html
# ⚠️ DESTRUCTIVE — changes are permanently lost, cannot undo this!

# Restore ALL changed files
git restore .

# SCENARIO 2: You staged a file but want to unstage it
# (keeps your changes in the file, just removes from staging area)

git restore --staged index.html
# ✅ SAFE — your changes are preserved, just not staged anymore

# Unstage all files
git restore --staged .
```

`git revert` — Safe Undo (Creates New Commit)

`git revert` is the **safe** way to undo because it doesn't delete history — it creates a new commit that undoes the changes.

```
# Undo a specific commit by its hash
git revert a3f91b2

# This creates a new commit like:
# Revert "Add homepage HTML structure"
#
# This reverts commit a3f91b2c4d...

# Undo the most recent commit
git revert HEAD

# Undo without opening the editor (auto-generate message)
git revert HEAD --no-edit


HOW git revert WORKS:
═══════════════════════════════════════════════════════

Before revert:
  ── [A] ── [B] ── [C]  ← HEAD (C is the broken commit)

After: git revert C
  ── [A] ── [B] ── [C] ── [C'] ← HEAD
                         Revert commit that undoes C

✅ History is preserved
✅ Safe to do on shared/pushed commits
✅ Team members can see exactly what was undone and why
```

`git reset` — Powerful Undo (Rewrites History)

`git reset` moves the branch pointer backward, effectively "erasing" commits.

```
# === git reset MODES ===

# --soft: Move HEAD back but keep changes staged
git reset --soft HEAD~1
# Use when: You want to undo the commit but keep the staged changes
# Changes are: Still staged, ready to re-commit

# --mixed (DEFAULT): Move HEAD back, unstage changes, keep files
git reset HEAD~1
# Use when: You want to undo the commit and unstage, but keep file changes
# Changes are: In working directory, not staged

# --hard: Move HEAD back and DISCARD all changes completely
git reset --hard HEAD~1
# Use when: You want to completely eliminate a commit and its changes
# Changes are: GONE FOREVER ⚠️


git reset MODES COMPARISON:
═══════════════════════════════════════════════════════════

Original:  [A] ── [B] ── [C]  ← HEAD
           (with changes from C in files and staging)

After: git reset --soft HEAD~1
  [A] ── [B] ← HEAD
  Repository: C is gone
  Staging:    C's changes STILL STAGED  ← Ready to re-commit
  Files:      C's changes present

After: git reset HEAD~1 (--mixed)
  [A] ── [B] ← HEAD
  Repository: C is gone
  Staging:    Empty
  Files:      C's changes present  ← You decide what to do with them

After: git reset --hard HEAD~1
  [A] ── [B] ← HEAD
  Repository: C is gone
  Staging:    Empty
  Files:      C's changes GONE ⚠️ (CANNOT BE RECOVERED!)
```

Comparing `git revert` vs `git reset`

Feature	`git revert`	`git reset`
Modifies history?	❌ No (adds new commit)	✅ Yes (moves HEAD back)
Safe for shared repos?	✅ Always safe	❌ Dangerous if pushed
Recovery possible?	✅ Yes	⚠️ Hard mode: No
Shows the undo in log?	✅ Yes	❌ No trace
Best for	Undoing pushed commits	Local experimental cleanup

Practical Examples

```
# Scenario 1: You committed a file with a typo in the message
git commit -m "Ad homepage stlyes"  # Oops!

# Fix: Amend the last commit (before pushing)
git commit --amend -m "Add homepage styles"

# Scenario 2: You committed a file that should have been in .gitignore
echo "secret_password_123" > secrets.txt
git add secrets.txt
git commit -m "Add secrets"  # Big mistake!

# Fix (if not yet pushed):
git reset --soft HEAD~1    # Undo commit, keep file staged
git restore --staged secrets.txt  # Unstage
echo "secrets.txt" >> .gitignore  # Add to gitignore
git add .gitignore
git commit -m "Add gitignore for secrets file"

# Scenario 3: A commit broke the site and it's already pushed
git log --oneline
# abc1234 (HEAD) Add new payment system  ← broke everything
# def5678 Working version of homepage

git revert abc1234      # Creates a new commit undoing the bad changes
# Now the site works again, and the history shows exactly what happened
```

Common Mistakes ⚠️

- **Using `git reset --hard` on pushed commits** — This creates diverged histories and causes major problems for teammates
- **Forgetting that `git restore` is permanent** — There's no "undo" for `git restore`
- **Reverting a merge commit without `-m`** — Merge commits need a special flag:
  ```
  git revert -m 1 <merge-commit-hash>
  ```

Branching in Git

🎯 Learning Objectives

- Understand what a branch is and why branches are Git's superpower
- Visualize branching as parallel timelines
- Know when and why to create branches
- Understand the relationship between branches and commits

What is a Branch?

A branch is simply a **lightweight pointer to a commit**. That's it. When you commit, the branch pointer moves forward to your new commit.

UNDERSTANDING BRANCHES

```
Your project timeline without branches:
  [A] ── [B] ── [C] ── [D]
                        ↑
                       main (the pointer)

Creating a new branch (git branch feature):
  [A] ── [B] ── [C] ── [D]
                        ↑           ↑
                       main        feature  (both point to D)

After committing on feature branch:
  [A] ── [B] ── [C] ── [D] ── [E] ── [F]
                        ↑             ↑
                       main          feature

After committing on main too:
                                     [E] ── [F]  ← feature branch
                                    /
  [A] ── [B] ── [C] ── [D] ── [G]
                                    ↑
                                   main
```

Now they've diverged! You can work on both independently.

Why Branches are Git's Superpower

```
REAL-WORLD BRANCHING SCENARIOS
═══════════════════════════════════════════════════════════════

Scenario: You're building an e-commerce website

main branch:        The stable, working website (what customers see)
                    ──────────────────────────────────────────────

feature/cart:       New shopping cart feature (in development)
                    - - - - - - - - - - - - -

hotfix/price-bug:   Emergency fix for a pricing bug (urgent!)
                    ═══════════════════════

feature/dark-mode:  New dark mode (low priority, future release)
                    . . . . . . . . . . . . .

All three teams can work simultaneously without interfering!
When ready, each feature merges into main.
```

Creating and Switching Branches

🎯 Learning Objectives

- Create new branches with `git branch`
- Switch between branches with `git switch` and `git checkout`
- View all branches in a repository
- Understand the `HEAD` pointer

Branch Commands

```
# List all local branches
git branch
# Output (current branch shown with asterisk *):
# * main
#   feature/login

# List all branches including remote
git branch -a

# Create a new branch
git branch feature/user-profile

# Switch to a branch (modern syntax — preferred)
git switch feature/user-profile

# Create AND switch in one command (most common workflow)
git switch -c feature/user-profile
# -c means "create"

# Old syntax (still works, very commonly seen):
git checkout feature/user-profile        # switch to existing
git checkout -b feature/user-profile     # create and switch

# Delete a branch (after it's been merged)
git branch -d feature/user-profile

# Force delete (even if not merged — careful!)
git branch -D feature/user-profile

# Rename a branch
git branch -m old-name new-name
```

Understanding HEAD

```
THE HEAD POINTER
═══════════════════════════════════════════════════════════

HEAD is a special pointer that always shows WHERE YOU ARE.

Normally HEAD points to a branch (which points to a commit):

  HEAD → main → [commit D]

When you switch branches:
  git switch feature/login

  HEAD → feature/login → [commit F]

"Detached HEAD" state (rare, when you checkout a specific commit):
  git checkout a3f91b2

  HEAD → [commit A]  (HEAD points directly to a commit, not a branch)
  ⚠️ Any commits you make here won't belong to any branch!
  ⚠️ Usually you want to create a new branch to avoid losing work
```

Complete Branching Workflow Example

```
# === Scenario: Adding a contact page to a website ===

# 1. Start on main branch (always start from main)
git switch main
git status  # Make sure everything is clean

# 2. Create a feature branch
git switch -c feature/contact-page

# 3. Verify you're on the new branch
git branch
# * feature/contact-page
#   main

# 4. Do your work
cat > contact.html << 'EOF'
<!DOCTYPE html>
<html>
<body>
  <h1>Contact Us</h1>
  <form>
    <input type="email" placeholder="Your email">
    <textarea placeholder="Your message"></textarea>
    <button>Send</button>
  </form>
</body>
</html>
EOF

# 5. Commit on this branch
git add contact.html
git commit -m "Add contact page with email form"

# 6. Add more to this branch
echo "Contact page styles" >> contact.html
git add contact.html
git commit -m "Add contact page styles"

# 7. Check the log on this branch
git log --oneline
# b9c8d7e Add contact page styles
# a6f5e4d Add contact page with email form
# c3b2a1f (main) Previous commit on main  ← main is still here
```

```
# 8. Switch back to main — contact.html disappears!
git switch main
ls                  # contact.html is GONE from this branch
# (It's safely stored in the feature branch)

# 9. Switch back to feature branch
git switch feature/contact-page
ls                  # contact.html is back!
```

Hands-on Exercise 4: Working with Branches

```
# Task: Create a blog website with separate branches for each page

# Setup
mkdir blog-site
cd blog-site
git init
echo "# Blog Site" > README.md
git add README.md
git commit -m "Initialize blog project"

# Task 1: Create a branch for the homepage
git switch -c feature/homepage
cat > index.html << 'EOF'
<!DOCTYPE html>
<html>
<body><h1>My Blog</h1><p>Welcome!</p></body>
</html>
EOF
git add index.html
git commit -m "Add blog homepage"

# Task 2: Go back to main, create another branch
git switch main
git switch -c feature/about-page
cat > about.html << 'EOF'
<!DOCTYPE html>
<html>
<body><h1>About Me</h1><p>I love coding!</p></body>
</html>
EOF
git add about.html
git commit -m "Add about page"

# Task 3: View all branches
git branch
# You should see:
#   feature/about-page ← current
#   feature/homepage
#   main

# Task 4: Visualize the branch structure
git log --oneline --graph --all
```

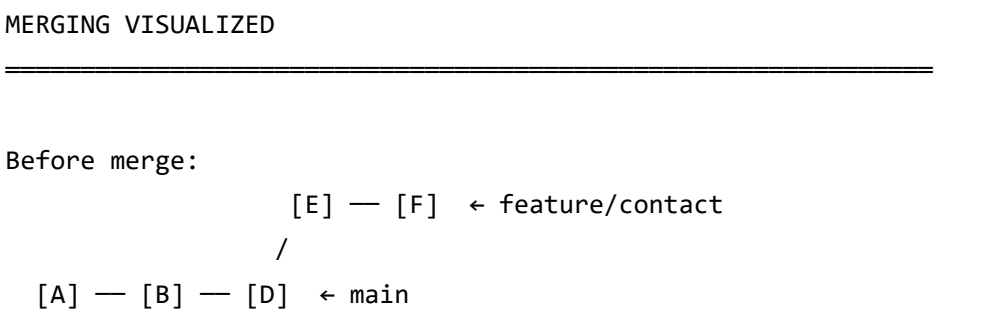
Basic Merging Techniques

🎯 Learning Objectives

- Understand what merging means in Git
- Perform a fast-forward merge
- Perform a three-way merge
- Understand and resolve basic merge conflicts

What is Merging?

Merging combines the work from two branches into one.



Two Types of Merges

Type 1: Fast-Forward Merge (the simple case)

```
FAST-FORWARD MERGE
═══════════════════════════════════════════════════

Condition: main has no new commits since branch was created

Before:
  [A] ── [B] ── [C]  ← main
                 \
                 [D] ── [E]  ← feature

After merge:
  [A] ── [B] ── [C] ── [D] ── [E]  ← main (feature)

No merge commit needed! main just "fast forwards" to feature's tip.
```

```
# Performing a fast-forward merge:
git switch main
git merge feature/contact-page

# Output:
# Updating c3b2a1f..b9c8d7e
# Fast-forward
#  contact.html | 10 +++++++++
#  1 file changed, 10 insertions(+)
#  create mode 100644 contact.html
```

Type 2: Three-Way Merge (when both branches have diverged)

```
THREE-WAY MERGE
═══════════════════════════════════════════════════

Condition: Both main and feature have new commits

Before:
                  [D] ── [E]  ← feature
                 /
  [A] ── [B] ── [C]
                 \
                 [F] ── [G]  ← main (new commits here!)

Git looks at 3 points:
  1. The common ancestor: [C]
  2. The feature tip: [E]
  3. The main tip: [G]

After merge:
                  [D] ── [E]
                 /           \
  [A] ── [B] ── [C]            [H]  ← main (merge commit)
                 \           /
                 [F] ── [G]
```

```
# Three-way merge happens automatically when branches have diverged
git switch main
git merge feature/contact-page

# Output:
# Merge made by the 'recursive' strategy.
#  contact.html | 10 +++++++++
#  1 file changed, 10 insertions(+)
```

Handling Merge Conflicts

A conflict happens when both branches changed the same part of the same file.

```
MERGE CONFLICT SCENARIO
═══════════════════════════════════════════════════

File on main branch:
  <h1>Hello World</h1>

Same file on feature branch:
  <h1>Hello Everyone</h1>

Git can't decide which to keep! → MERGE CONFLICT


# When a conflict occurs, Git tells you:
git merge feature/new-heading
# CONFLICT (content): Merge conflict in index.html
# Automatic merge failed; fix conflicts and then commit the result.

# The conflicted file looks like this:
cat index.html
# <<<<<<< HEAD              ← Your current branch's version starts here
# <h1>Hello World</h1>
# =======                   ← Separator
# <h1>Hello Everyone</h1>
# >>>>>>> feature/new-heading ← Incoming branch's version ends here


# Steps to resolve a conflict:

# 1. Open the conflicted file in your editor
# 2. Remove the conflict markers and decide what to keep

# For example, edit index.html to be:
# <h1>Hello World and Everyone</h1>
# (or whichever version you want to keep)

# 3. Stage the resolved file
git add index.html

# 4. Complete the merge
git commit -m "Merge feature/new-heading — kept combined greeting"

# Check the result
git log --oneline --graph
```

Hands-on Exercise 5: Your First Merge

```
# Using the blog-site from Exercise 4

# Task: Merge feature/homepage into main

# Step 1: Make sure main is up to date
git switch main

# Step 2: Merge the homepage feature
git merge feature/homepage
# Should be a fast-forward merge

# Step 3: Verify index.html is now in main
ls
# Should see: README.md  index.html

# Step 4: View the history
git log --oneline --graph --all

# Challenge: Merge feature/about-page too
git merge feature/about-page

# Step 5: Clean up merged branches
git branch -d feature/homepage
git branch -d feature/about-page

# View final state
git branch
# Should only show: * main
```

Best Practices for Branching & Merging

1. **Always merge into main from an up-to-date main** — Run `git pull` before creating branches
2. **Keep branches short-lived** — Long-lived branches become harder to merge
3. **Use descriptive branch names** — `feature/user-auth` not `mywork` or `test`
4. **Delete merged branches** — Keep the repository clean
5. **Never work directly on main** — Always use feature branches

Quick Reference — Branching & Merging

`git branch`	List local branches
`git branch -a`	List all branches (including remote)
`git branch <name>`	Create a branch
`git switch <name>`	Switch to a branch
`git switch -c <name>`	Create and switch (most common)
`git merge <branch>`	Merge a branch into current branch
`git branch -d <name>`	Delete a merged branch
`git branch -D <name>`	Force delete a branch
`git log --graph --oneline`	Visualize branch history

Additional Resources for Day 1

- **Official Git Documentation:** git-scm.com/doc
- **Interactive Learning:** learngitbranching.js.org
- **Git Cheat Sheet:** education.github.com/git-cheat-sheet
- **Pro Git Book (Free):** git-scm.com/book

Section 4.0

Git Branching, Merging, and Collaborative Features

```
╔═══════════════════════════════════════════════════════╗
║                                                       ║
║   DAY 2: Advanced Git — Branching, Merging & Teamwork ║
║                                                       ║
║   By now you can:                                     ║
║   ✅ Initialize repositories                          ║
║   ✅ Stage and commit changes                         ║
║   ✅ View history and undo mistakes                   ║
║   ✅ Create branches and do basic merges              ║
║                                                       ║
║   Today you'll master:                                ║
║   🎯 Professional branching strategies                ║
║   🎯 Complex merging and conflict resolution          ║
║   🎯 Rebase — Git's most powerful rewriting tool      ║
║   🎯 Working with remote repositories                 ║
║                                                       ║
╚═══════════════════════════════════════════════════════╝
```

Section 4.1

Assignment 2 & Advanced Branching

Advanced Branching Strategies

🎯 Learning Objectives

- Understand professional branching strategies used by real teams
- Implement Git Flow — the industry-standard workflow
- Use feature branches and hotfix branches correctly

- Know how to manage multiple simultaneous features

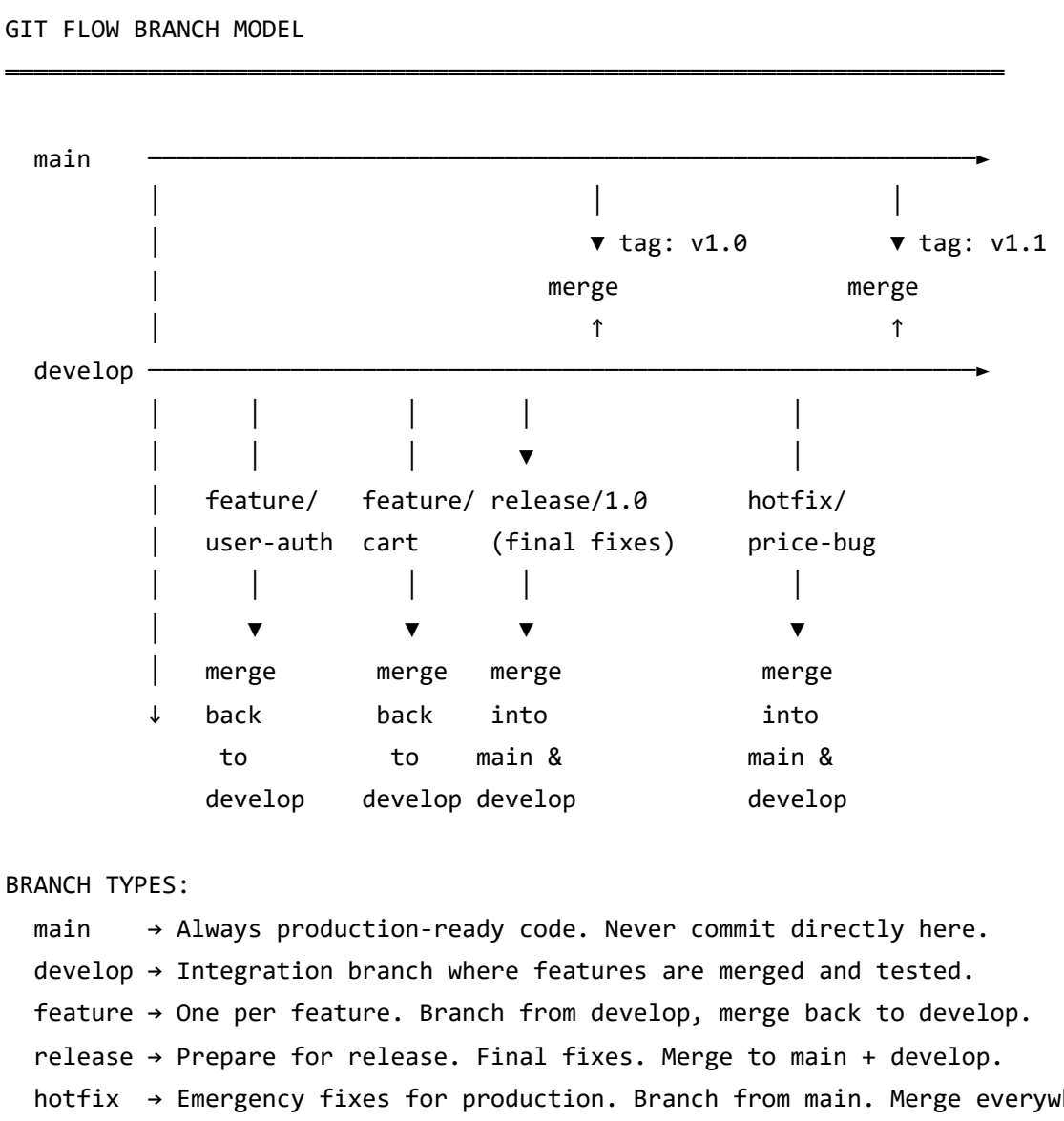
Why Branching Strategies Matter

Without a branching strategy, a team of 10 developers becomes chaos. Everyone merges when they feel like it, production breaks regularly, and nobody knows what's "the working version."

A **branching strategy** is an agreement about how the team will use branches.

Git Flow — The Industry Standard

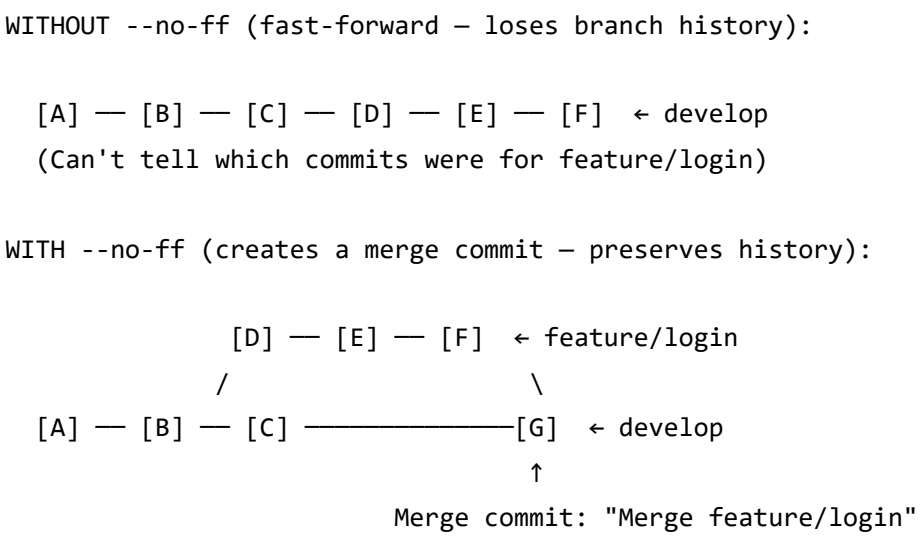
Git Flow defines specific branch types with specific purposes:



Feature Branches

```
# Feature Branch Naming Convention
feature/user-authentication
feature/shopping-cart
feature/dark-mode
feature/TICKET-123-payment-integration   # Often includes ticket number

# === Creating a Feature Branch ===

# 1. Always start from the latest develop (or main)
git switch develop
git pull origin develop      # Get the latest code

# 2. Create and switch to feature branch
git switch -c feature/user-authentication

# 3. Do your work with regular commits
# ... edit files ...
git add .
git commit -m "Add login form HTML and CSS"

# ... more work ...
git add .
git commit -m "Add form validation JavaScript"

# ... more work ...
git add .
git commit -m "Connect login form to API endpoint"

# 4. When feature is complete, merge back
git switch develop
git merge feature/user-authentication --no-ff   # --no-ff preserves branch history

# 5. Clean up
git branch -d feature/user-authentication
git push origin develop
```

The `--no-ff` Flag (No Fast-Forward)



Hotfix Branches — Emergency Production Fixes

```
# === The Hotfix Workflow ===

# SCENARIO: Your production website has a critical bug RIGHT NOW
# Customers can't checkout because of a pricing calculation error
# You need to fix it IMMEDIATELY without waiting for the next release

# 1. Create hotfix from main (not develop — develop has unfinished features!)
git switch main
git pull origin main          # Make sure you have the latest production code
git switch -c hotfix/fix-price-calculation

# 2. Fix the bug
# Edit the pricing calculation in checkout.js
git add checkout.js
git commit -m "Fix: Correct rounding error in price calculation"
# The rounding used parseInt() which strips decimals - replaced with toFixed(2)

# 3. Merge into MAIN first (this fixes production)
git switch main
git merge hotfix/fix-price-calculation --no-ff
git tag -a v1.0.1 -m "Hotfix: Correct price calculation rounding"
git push origin main --tags

# 4. ALSO merge into develop (so the fix is in future releases too)
git switch develop
git merge hotfix/fix-price-calculation --no-ff
git push origin develop

# 5. Clean up
git branch -d hotfix/fix-price-calculation

# RESULT: Production is fixed within minutes!
```

```
HOTFIX FLOW:
═══════════════════════════════════════════════════════════

main     ──[v1.0]──────────────────────[v1.0.1]──►
               \                        /
hotfix          ──[fix pricing bug]──
                                      \
develop  ──────────────────────────────[merged fix]──►

The hotfix is applied to both branches so nothing is lost!
```

Release Branches

```
# === The Release Workflow ===

# When develop has all the features for v2.0, create a release branch
git switch develop
git switch -c release/2.0

# Now ONLY bug fixes go here (no new features)
# Fix discovered during testing:
git add login.js
git commit -m "Fix: Handle empty username in login validation"

# When ready, merge to main
git switch main
git merge release/2.0 --no-ff
git tag -a v2.0 -m "Release version 2.0"
git push origin main --tags

# Also merge back to develop
git switch develop
git merge release/2.0 --no-ff
git push origin develop

# Clean up
git branch -d release/2.0
```

Simplified Trunk-Based Development (Alternative)

For smaller teams or CI/CD pipelines, a simpler approach:

```
TRUNK-BASED DEVELOPMENT
═══════════════════════════════════════════════════════════

main (trunk):  [A]──[B]──[C]──[D]──[E]──[F]──►
                       ↑    ↑         ↑
                  Feature A  Feature B  Feature C
                  merged     merged     merged
                  (all small, short-lived branches)

Rules:
- Branches live max 1-2 days
- Deploy to production frequently (daily or per-commit)
- Use feature flags to hide incomplete features
- Excellent for experienced teams with good test coverage
```

Choosing the Right Strategy

Factor	Git Flow	Trunk-Based
Team size	Medium to large (5+)	Small to medium
Release frequency	Planned releases (weekly/monthly)	Continuous deployment
Testing maturity	Less automated testing OK	Requires strong test suite
Product type	Apps with distinct versions	Web services/SaaS
Complexity	Higher — more branches	Lower — simpler

Complex Merging Techniques

🎯 Learning Objectives

- Handle complex merge conflicts with confidence
- Use merge strategies and options effectively
- Understand octopus merges
- Use `git mergetool` for visual conflict resolution

Resolving Complex Merge Conflicts

```
COMPLEX CONFLICT SCENARIO
═══════════════════════════════════════════════════════════════

Developer A changed lines 10-15 of app.js (added error handling)
Developer B changed lines 12-18 of app.js (added new feature)
Lines 12-15 were changed by BOTH developers → CONFLICT!
```

```
# When you encounter a complex conflict:
git merge feature/payment-system
# CONFLICT (content): Merge conflict in app.js
# CONFLICT (content): Merge conflict in checkout.html
# Automatic merge failed; fix conflicts and then commit the result.

# Check which files have conflicts
git status
# Both modified: app.js
# Both modified: checkout.html

# See the full diff of conflicts
git diff

# Open conflicted file and understand the markers:
cat app.js
```

```
// WHAT A COMPLEX CONFLICT LOOKS LIKE:

function processPayment(amount) {
<<<<<<< HEAD
  // Developer A's version (current branch)
  if (amount <= 0) {
    throw new Error('Amount must be positive');
  }
  const result = chargeCard(amount);
  return result;
=======
  // Developer B's version (incoming branch)
  const taxRate = 0.08;
  const total = amount * (1 + taxRate);
  const result = chargeCard(total);
  logTransaction(total);
  return result;
>>>>>>> feature/payment-system
}


// AFTER MANUAL RESOLUTION (combining both changes):

function processPayment(amount) {
  // Combined: A's validation + B's tax calculation + B's logging
  if (amount <= 0) {
    throw new Error('Amount must be positive');
  }
  const taxRate = 0.08;
  const total = amount * (1 + taxRate);
  const result = chargeCard(total);
  logTransaction(total);
  return result;
}


# After manually resolving all conflicts:
git add app.js
git add checkout.html
git commit -m "Merge feature/payment-system - combine validation and tax logic"
```

Using `git mergetool`

For complex conflicts, a visual tool helps enormously:

```
# Set up a merge tool (one-time setup)
git config --global merge.tool vimdiff      # Terminal tool
# OR
git config --global merge.tool vscode       # VS Code

# Launch the tool when you have conflicts
git mergetool

# VS Code shows three panels:
# LEFT:   Your current branch's version
# CENTER: The common ancestor (what it was before either change)
# RIGHT:  The incoming branch's version
# BOTTOM: The final result you're editing

# After resolving, close the editor and run:
git commit
```

Merge Strategies

```
# Default (recursive/ort) - handles most situations
git merge feature/new-ui

# Keep OUR version when conflicts occur (current branch wins)
git merge -X ours feature/new-ui

# Keep THEIR version when conflicts occur (incoming branch wins)
git merge -X theirs feature/new-ui

# Abort a merge mid-way (return to pre-merge state)
git merge --abort

# Squash all commits into one before merging (clean history)
git merge --squash feature/new-ui
git commit -m "Add new UI (squashed feature branch)"
```

```
SQUASH MERGE EXPLAINED:
═══════════════════════════════════════════════════════════════

Feature branch:  [A]──[B]──[C]──[D]──[E]  ← 5 messy commits

After git merge --squash:
main:  ──────────────────────────────────[F]  ← ONE clean commit
                                         ↑
                                  Contains all changes from A-E
                                  but looks like one commit

Use when: Feature branch has lots of "WIP" and "fix typo" commits
          and you want a clean merge commit
```

Resolving Complex Merge Conflicts

A Real-World Conflict Resolution Protocol

```
CONFLICT RESOLUTION WORKFLOW
═══════════════════════════════════════════════════════

Step 1: Don't panic!
        git status       ← see which files conflict

Step 2: Understand the conflict
        git log --merge  ← see the commits that caused conflict
        git diff --merge ← see all conflict diffs at once

Step 3: For each conflicted file:
        a. Open the file
        b. Find <<<<<, =====, >>>>> markers
        c. Understand WHAT each version does
        d. Decide: keep A, keep B, or combine both
        e. Edit the file (remove markers!)
        f. Test that the change works

Step 4: Stage resolved files
        git add <resolved-file>

Step 5: Verify all conflicts resolved
        git status      ← should show no "both modified" files

Step 6: Complete the merge
        git commit      ← Git provides a default merge message
```

Conflict Prevention Strategies

```
HOW TO AVOID CONFLICTS IN THE FIRST PLACE
═══════════════════════════════════════════

1. COMMUNICATE with teammates
   → "I'm working on checkout.js this week"
   → Use a ticket/task system to track who's doing what

2. KEEP BRANCHES SHORT-LIVED
   → Feature done in 2 days → 2 days of divergence → few conflicts
   → Feature takes 3 weeks → 3 weeks of divergence → many conflicts

3. PULL FREQUENTLY from main/develop
   git pull origin develop    ← Do this daily!

4. SMALL, FOCUSED CHANGES
   → Each branch changes only the files it needs to
   → Don't "clean up" or reformat files you're not working on

5. MODULAR CODE STRUCTURE
   → Split large files into smaller ones
   → Small files = less chance two people edit same file
```

Rebase vs Merge

🎯 Learning Objectives

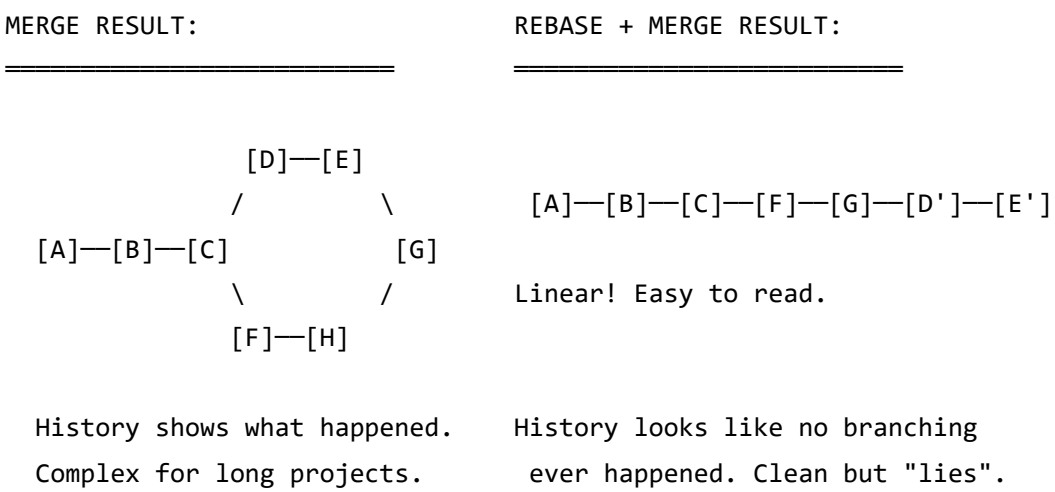
- Understand what `git rebase` does
- Know when to use rebase vs. merge
- Perform an interactive rebase to clean up history
- Understand the Golden Rule of rebasing

What is Rebase?

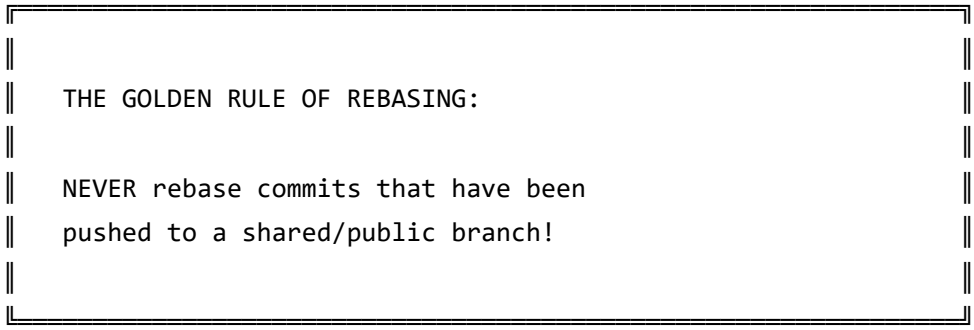
Rebase "replays" your commits on top of another branch, making it look like you branched off the latest version.

```
REBASE VISUALIZED
═══════════════════════════════════════════════════════════════

BEFORE REBASE:
                  [D] ── [E]  ← feature
                 /
  [A] ── [B] ── [C] ── [F] ── [G]  ← main
         ↑
  feature was created here, main has moved on

AFTER: git rebase main (from feature branch):

  [A] ── [B] ── [C] ── [F] ── [G] ── [D'] ── [E']  ← feature
                                       ↑
                              D and E are RECREATED on top of G
                              (new commits, new SHA hashes)

AFTER MERGING (fast-forward):

  [A] ── [B] ── [C] ── [F] ── [G] ── [D'] ── [E']  ← main

  CLEAN, LINEAR HISTORY! No merge commits needed.


# === The Rebase Workflow ===

# Scenario: You're on feature/login, and main has moved forward

git switch feature/login

# Rebase: "replay my commits on top of latest main"
git rebase main

# If conflicts occur during rebase:
# 1. Resolve the conflict in the file
# 2. git add <resolved-file>
# 3. git rebase --continue   (do NOT git commit during rebase!)
# 4. Repeat for each commit that conflicts

# Abort a rebase
git rebase --abort    # Returns to pre-rebase state

# After rebase, merge is a simple fast-forward:
git switch main
git merge feature/login   # Fast-forward, no merge commit!
```

Rebase vs. Merge: Side-by-Side



The Golden Rule of Rebasing ⚠️



WHY? Rebase RECREATES commits with new SHA hashes.
If teammates already have the old commits, their history
diverges from yours and causes major problems.

SAFE TO REBASE:	NEVER REBASE:
✅ Local feature	❌ main
✅ Unpushed commits	❌ develop
✅ Your own branches	❌ Any branch others have pulled

Interactive Rebase — Cleaning Up History

Interactive rebase lets you rewrite, combine, rename, and reorder commits before sharing.

```
# Interactively rebase the last 4 commits
git rebase -i HEAD~4

# This opens your editor with:
pick a1b2c3d Add login form HTML
pick b2c3d4e wip
pick c3d4e5f fix typo
pick d4e5f6g Add login form CSS and JS

# Commands you can use (change "pick" to these):
# pick   = keep commit as-is
# reword = keep commit, but change the message
# edit   = pause and amend the commit
# squash = combine with the PREVIOUS commit (merge messages)
# fixup  = combine with previous commit (discard message)
# drop   = completely remove this commit


# EXAMPLE: Clean up messy development commits before merging

# Before rebase -i:
# d4e5f6g Add CSS - FINAL FINAL v2  ← messy!
# c3d4e5f Actually fix the bug       ← messy!
# b2c3d4e Fix bug (forgot file)      ← messy!
# a1b2c3d Add login form             ← good

# Edit the rebase file to:
pick a1b2c3d Add login form
fixup b2c3d4e Fix bug (forgot file)      ← combine with next
squash c3d4e5f Actually fix the bug       ← combine with next
reword d4e5f6g Add CSS - FINAL FINAL v2  ← rename this

# After rebase, history becomes:
# [clean] Add login form CSS and styles   ← 3 commits become 1 named commit!
# [clean] Add login form                  ← original

# MUCH CLEANER and easier to understand!
```

Comparing Rebase and Merge Approaches

Situation	Use Merge	Use Rebase
Feature → main on shared repo	✅ Standard	⚠️ After team agrees
Updating local feature with main	✅ Fine	✅ Preferred (cleaner)

Situation	Use Merge	Use Rebase
Cleaning up personal commit history	❌	✅ Perfect use case
Preserving exact project history	✅	❌ Rewrites history
Public/shared branches	✅ Always	❌ Never

Assignment 2: Git Workflow Practice

Scenario: You're a developer on a team building a task management application.

```
# === ASSIGNMENT 2 TASKS ===

# SETUP
mkdir taskapp
cd taskapp
git init
git switch -c develop    # Create develop branch

# Create initial project structure
mkdir src tests docs
echo "# TaskApp - Team Task Manager" > README.md
echo "version=1.0" > package.json
git add .
git commit -m "Initialize TaskApp project structure"

# TASK 1: Create a feature branch and implement a feature
git switch -c feature/task-creation

cat > src/task.js << 'EOF'
// Task Creation Module
class Task {
  constructor(title, dueDate, assignee) {
    this.id = Date.now();
    this.title = title;
    this.dueDate = dueDate;
    this.assignee = assignee;
    this.status = 'pending';
    this.createdAt = new Date();
  }

  complete() {
    this.status = 'completed';
    this.completedAt = new Date();
  }
}

module.exports = Task;
EOF

git add src/task.js
git commit -m "Add Task class with creation and completion logic"

# Add tests
cat > tests/task.test.js << 'EOF'
const Task = require('../src/task');
```

```
  describe('Task', () => {
    test('creates task with correct properties', () => {
      const task = new Task('Write report', '2024-02-01', 'Alice');
      expect(task.title).toBe('Write report');
      expect(task.status).toBe('pending');
    });

    test('marks task as completed', () => {
      const task = new Task('Write report', '2024-02-01', 'Alice');
      task.complete();
      expect(task.status).toBe('completed');
    });
  });
  EOF

  git add tests/task.test.js
  git commit -m "Add unit tests for Task class"

  # TASK 2: Merge feature back to develop
  git switch develop
  git merge feature/task-creation --no-ff -m "Merge feature/task-creation: Add Task model with te
  git branch -d feature/task-creation

  # TASK 3: Create and resolve a hotfix
  # Simulate a bug being discovered
  git switch -c hotfix/fix-task-id
  # Fix: Use UUID instead of timestamp for unique IDs
  cat > src/task.js << 'EOF'
  // Task Creation Module
  let taskCounter = 1;   // Simple counter for reliable unique IDs

  class Task {
    constructor(title, dueDate, assignee) {
      this.id = `TASK-${taskCounter++}`;  // Fixed: predictable IDs
      this.title = title;
      this.dueDate = dueDate;
      this.assignee = assignee;
      this.status = 'pending';
      this.createdAt = new Date();
    }

    complete() {
      this.status = 'completed';
      this.completedAt = new Date();
    }
  }
```

```
module.exports = Task;
EOF

git add src/task.js
git commit -m "Fix: Use sequential counter for reliable task IDs (replaces timestamp)"

# Merge hotfix to develop
git switch develop
git merge hotfix/fix-task-id --no-ff
git branch -d hotfix/fix-task-id

# TASK 4: Interactive rebase practice
git switch -c feature/task-list
echo "// Task list feature - draft" > src/taskList.js
git add src/taskList.js
git commit -m "wip task list"

echo "// TaskList implementation" > src/taskList.js
echo "class TaskList { constructor() { this.tasks = []; } }" >> src/taskList.js
git add src/taskList.js
git commit -m "more wip"

echo "  addTask(task) { this.tasks.push(task); }" >> src/taskList.js
git add src/taskList.js
git commit -m "ok done i think"

# Clean up with interactive rebase
git rebase -i HEAD~3
# In the editor: squash the 2nd and 3rd into the first, reword the message

# View the final clean history
git log --oneline --graph --all
```

Expected Outcome: A clean git history demonstrating proper use of feature branches, hotfixes, and interactive rebase.

Section 4.2

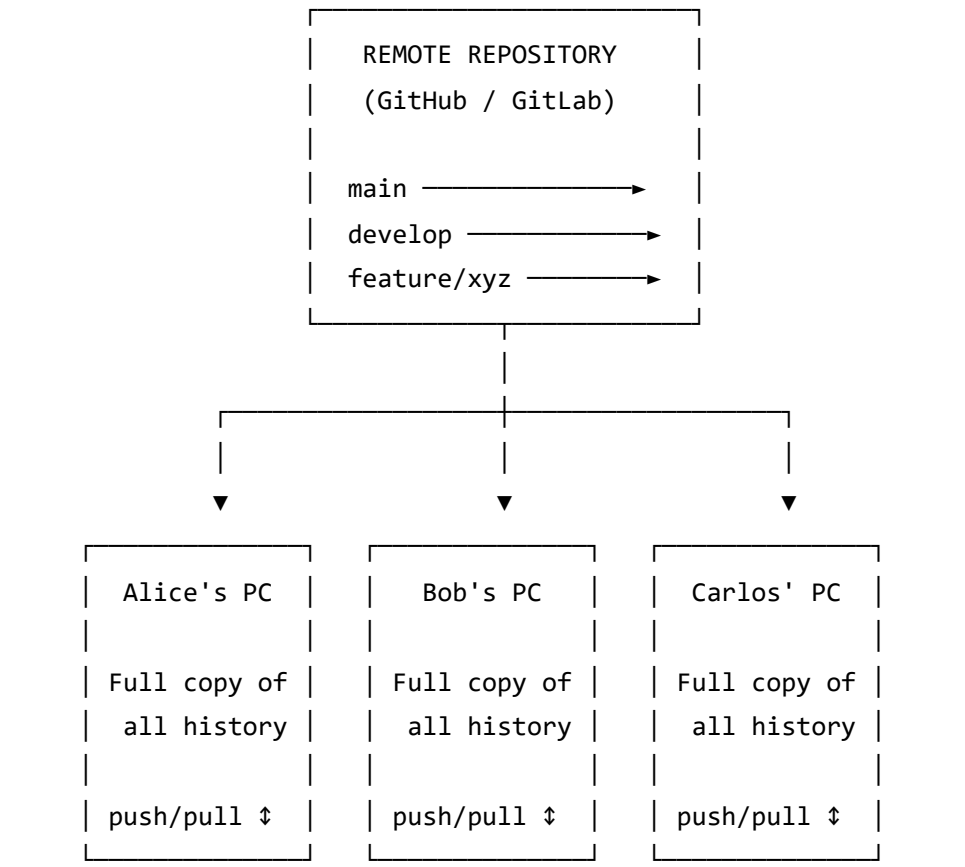
Collaborative Development with Git

Collaborative Workflows in Git

🎯 Learning Objectives

- Understand centralized vs. distributed collaboration models
- Set up remote repositories (GitHub/GitLab)
- Use `git push` and `git pull` effectively
- Handle remote branch tracking

The Collaboration Picture



Each developer:

1. Pulls latest code from remote
2. Creates a branch and does their work
3. Pushes their branch to remote
4. Creates a Pull Request for code review
5. After approval, merge into main/develop

Setting Up Remote Repositories

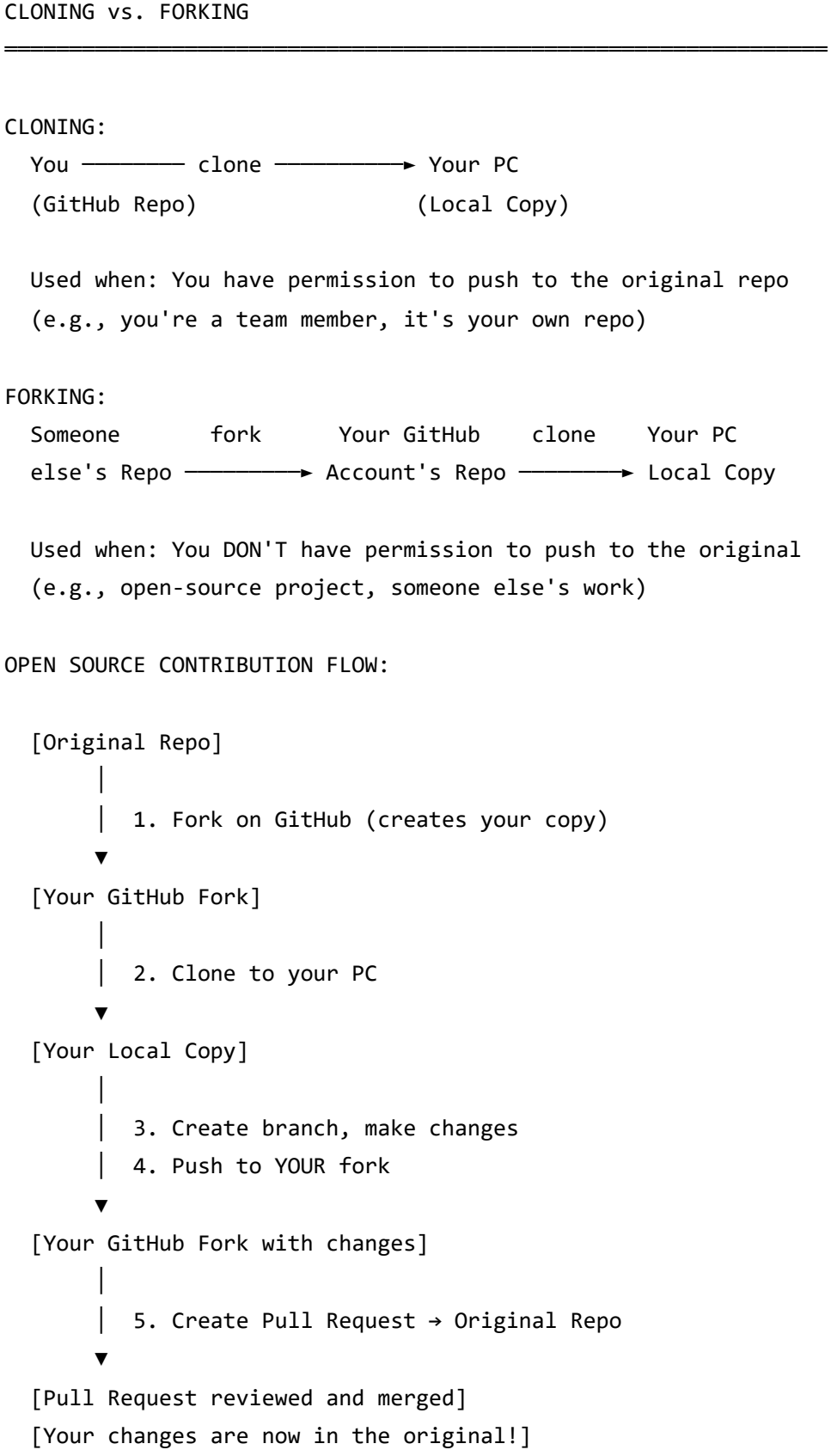
```
# === Option 1: Start from GitHub and clone ===

# 1. Create a new repo on GitHub.com
# 2. Copy the repository URL (e.g., https://github.com/username/project.git)

# Clone the repository to your local machine
git clone https://github.com/username/my-project.git

# This automatically:
# - Creates a local copy of the entire repo
# - Sets up 'origin' as the remote name
# - Creates tracking for the main branch

# Navigate into the cloned folder
cd my-project

# === Option 2: Push existing local repo to GitHub ===

# 1. Create empty repo on GitHub (don't initialize with README)
# 2. In your local project:

git remote add origin https://github.com/username/my-project.git

# Verify the remote was added
git remote -v
# origin  https://github.com/username/my-project.git (fetch)
# origin  https://github.com/username/my-project.git (push)

# Push to GitHub for the first time
git push -u origin main
# -u sets up tracking (only needed first time)
```

Forking and Cloning Repositories

🎯 Learning Objectives

- Understand the difference between forking and cloning
- Know when to fork vs. when to clone
- Contribute to open-source projects via forks
- Keep a fork synchronized with the original

Cloning vs. Forking



The Cloning Process in Detail

```
# Basic clone
git clone https://github.com/username/project.git

# Clone into a specific folder name
git clone https://github.com/username/project.git my-project-name

# Clone a specific branch
git clone -b develop https://github.com/username/project.git

# After cloning, explore what you got:
cd project
git remote -v         # See the 'origin' remote
git branch -a         # See all branches (local and remote)
git log --oneline -5  # See recent history
```

Keeping Your Fork Updated

```
# When the original repository gets new commits,
# your fork falls behind. Here's how to sync:

# 1. Add the original repo as "upstream" remote (one-time setup)
git remote add upstream https://github.com/ORIGINAL-OWNER/original-project.git

# Verify you now have two remotes
git remote -v
# origin    https://github.com/YOUR-USERNAME/project.git (fetch)
# origin    https://github.com/YOUR-USERNAME/project.git (push)
# upstream  https://github.com/ORIGINAL-OWNER/project.git (fetch)
# upstream  https://github.com/ORIGINAL-OWNER/project.git (push)

# 2. Fetch updates from original (doesn't modify your code yet)
git fetch upstream

# 3. Merge upstream changes into your main
git switch main
git merge upstream/main

# 4. Push your updated main to your fork
git push origin main

# Now your fork is synchronized with the original!
```

Working with Branches Remotely

🎯 Learning Objectives

- Push local branches to remote repositories
- Track remote branches
- Fetch, pull, and push effectively
- Clean up remote branches

Understanding Remote Tracking Branches

```
REMOTE TRACKING BRANCHES
═══════════════════════════════════════════════════════════════

When you clone or fetch, Git creates "remote tracking branches":

Local branches:              Remote tracking branches:
  main                         origin/main
  develop                      origin/develop
  feature/login                origin/feature/login

Remote tracking branches are READ-ONLY snapshots of the remote.
They update only when you fetch/pull.

  Your local: main ──── push/pull ──► Remote: origin/main
              (editable)                      (read-only snapshot)

              origin/main = your local record of what's on the server
```

Pushing and Pulling Branches

```
# === PUSHING ===

# Push a branch to remote (first time — sets up tracking)
git push -u origin feature/user-auth
# -u = --set-upstream (only needed once per branch)

# After tracking is set up, just:
git push

# Push to a remote branch with a different name (rarely needed)
git push origin local-branch:remote-branch-name

# Push all local branches at once (use carefully)
git push --all origin

# Force push (DANGEROUS — only use on your OWN branches, never main!)
git push --force-with-lease origin feature/my-branch
# --force-with-lease is safer than --force (fails if remote has new commits)

# === PULLING ===

# Pull latest changes (fetch + merge)
git pull

# Pull from a specific remote and branch
git pull origin develop

# Pull with rebase instead of merge (keeps cleaner history)
git pull --rebase origin main

# Fetch only (download changes without merging)
git fetch origin
git fetch --all    # Fetch from all remotes

# After fetch, you see what changed without applying it:
git log HEAD..origin/main --oneline   # What's on remote but not local
git log origin/main..HEAD --oneline   # What's on local but not remote
```

Remote Branch Management

```
# === VIEWING REMOTE BRANCHES ===

# List all remote branches
git branch -r

# List all branches (local and remote)
git branch -a

# See which local branches track which remote branches
git branch -vv
# Output:
# * main          abc1234 [origin/main] Last commit message
#   feature/auth  def5678 [origin/feature/auth: ahead 2] Add auth logic
#   develop       ghi9012 [origin/develop: behind 3] Setup dev environment

# === CHECKING OUT A REMOTE BRANCH ===

# Fetch all remote branches
git fetch origin

# Create a local branch tracking a remote branch
git switch --track origin/feature/new-dashboard
# Shorthand (Git figures out the remote):
git switch feature/new-dashboard   # If only one remote has it

# === DELETING REMOTE BRANCHES ===

# Delete a branch on the remote server
git push origin --delete feature/completed-feature

# Prune: Remove local references to deleted remote branches
git fetch --prune
# Or configure Git to always prune:
git config --global fetch.prune true
```

The Daily Collaboration Flow

```
YOUR DAILY GIT ROUTINE (as a team developer)
═══════════════════════════════════════════════════════════

MORNING — Start of day:
  1. git switch main
  2. git pull origin main          ← Get overnight changes
  3. git switch -c feature/my-task ← Create new branch for today's work

DURING THE DAY — Working:
  4. Edit files...
  5. git add .
  6. git commit -m "Clear description"
  7. Repeat 4-6 regularly

END OF DAY — Share your work:
  8. git push -u origin feature/my-task  ← Push to remote (backup + visibility)

NEXT DAY — Keep branch current:
  9. git switch main
  10. git pull origin main
  11. git switch feature/my-task
  12. git rebase main    ← Incorporate overnight changes from team
  13. Continue working...

FEATURE COMPLETE:
  14. git push origin feature/my-task
  15. Create Pull Request on GitHub
  16. Request code review from teammates
  17. Address feedback, push more commits
  18. PR approved → Merge!
  19. git branch -d feature/my-task    ← Clean up locally
  20. git push origin --delete feature/my-task  ← Clean up remotely
```

Hands-on Exercise 6: Collaborative Workflow Simulation

```
# Simulate collaboration by using two different local directories
# as if they were two different developers

# === DEVELOPER ALICE (terminal window 1) ===

mkdir /tmp/alice-project
cd /tmp/alice-project

# Create a bare repo to simulate GitHub
git init --bare /tmp/central-repo.git

# Alice clones it
git clone /tmp/central-repo.git .

# Alice sets up the project
echo "# Team Project" > README.md
echo "Initial structure" > app.js
git add .
git commit -m "Initialize project"
git push origin main

# === DEVELOPER BOB (terminal window 2) ===

mkdir /tmp/bob-project
cd /tmp/bob-project
git clone /tmp/central-repo.git .

# Bob creates a feature branch
git switch -c feature/user-module
echo "// User module" > user.js
git add user.js
git commit -m "Add user module"
git push -u origin feature/user-module

# === BACK TO ALICE ===
cd /tmp/alice-project
git fetch origin
git branch -a    # Alice can see Bob's branch!

# Alice also works
git switch -c feature/admin-panel
echo "// Admin panel" > admin.js
git add admin.js
git commit -m "Add admin panel"
```

```
git push -u origin feature/admin-panel

# === MERGING (Alice merges Bob's work) ===
git switch main
git pull origin main
git merge origin/feature/user-module
git push origin main

# Bob pulls Alice's merged work
cd /tmp/bob-project
git switch main
git pull origin main    # Bob now has Alice's admin panel too!
```

Section 4.3

Code Review and Collaboration Practices

Utilizing Pull Requests

🎯 Learning Objectives

- Understand the purpose and anatomy of a Pull Request
- Create effective Pull Requests that are easy to review
- Conduct thorough and constructive code reviews
- Respond to and address code review feedback
- Use PR templates and automation

What is a Pull Request?

A **Pull Request (PR)** — called a **Merge Request (MR)** in GitLab — is a formal request to merge code from one branch into another. It's the primary mechanism for code review and team collaboration.

```
THE PULL REQUEST WORKFLOW
═══════════════════════════════════════════════════════════════

  Developer                        Team
  _________                        ____

  1. Creates branch
  2. Writes code
  3. Pushes branch          ──────────►
  4. Opens PR               ──────────►  5. Team gets notified
                                         6. Reviewers read the code
                                         7. Leave comments/feedback
                            ◄──────────  8. Request changes
  9. Make changes
  10. Push more commits     ──────────►  11. Review again
                                         12. Approve ✅
                            ◄──────────
  13. PR is merged!
  14. Branch deleted

  BENEFITS:
  ✅ Every change is reviewed before it hits main
  ✅ Knowledge sharing across the team
  ✅ Discussion preserved in PR history
  ✅ Automated tests run before merge
  ✅ Trail of WHY decisions were made
```

Creating and Managing Pull Requests

Creating an Effective PR

Step 1: Prepare Your Branch

```
# Make sure your branch is current and clean
git switch feature/user-authentication
git rebase main          # Base on latest main
git push origin feature/user-authentication
```

Step 2: On GitHub, Open a Pull Request

Navigate to your repository on GitHub. GitHub will often show a banner "Compare & pull request" for recently pushed branches. Click it.

Fill in the PR template:

```
## Summary
Implements JWT-based user authentication to replace session cookies.
Closes #47 (the GitHub issue this addresses)

## What Changed
- Added JWT middleware to Express server
- Updated login/logout endpoints to use tokens
- Added token refresh logic with 24h expiry
- Updated all protected routes to validate JWT

## Why This Approach
JWT tokens allow stateless auth, which is better for our planned
horizontal scaling. Session cookies require sticky sessions or a
shared Redis store.

## Testing
- [x] Unit tests for JWT generation and validation
- [x] Integration tests for login/logout flow
- [x] Tested manually on development server
- [x] Token refresh tested with expired tokens

## Screenshots (if UI changes)
[Attach screenshots here]

## Checklist
- [x] Code follows project style guide
- [x] Tests added for new functionality
- [x] Documentation updated
- [x] No console.log statements left in
- [x] Environment variables documented
```

Step 3: Assign Reviewers

- Assign 1-2 team members as reviewers
- Tag relevant people (e.g., `@backend-team` for API changes)
- Link to the relevant issue/ticket

Conducting a Code Review

As a **reviewer**, your job is to:

CODE REVIEW CHECKLIST

═══

CORRECTNESS:

- ☐ Does the code actually do what the PR says?
- ☐ Are edge cases handled? (empty inputs, nulls, errors)
- ☐ Are there any obvious bugs?

READABILITY:

- ☐ Can you understand what the code does without the author explaining?
- ☐ Are variable/function names descriptive?
- ☐ Are complex sections commented?

SECURITY:

- ☐ Is user input validated/sanitized?
- ☐ Are there SQL injection risks? (use parameterized queries)
- ☐ Are secrets or API keys hardcoded? (never!)
- ☐ Is authorization checked properly?

PERFORMANCE:

- ☐ Are there obvious inefficiencies? (e.g., database call inside a loop)
- ☐ Are database queries optimized?
- ☐ Is caching used where appropriate?

TESTING:

- ☐ Are there tests for new functionality?
- ☐ Do tests cover happy path AND error cases?
- ☐ Would a bug in this code be caught by the tests?

DESIGN:

- ☐ Does this fit with the overall system architecture?
- ☐ Is there code duplication that could be abstracted?
- ☐ Does this follow SOLID principles / project conventions?

Writing Good Code Review Comments

```
THE CODE REVIEW COMMENT SPECTRUM
═══════════════════════════════════════════════════════════════

❌ UNHELPFUL COMMENTS:
  "This is wrong."
  "Why did you do it this way??"
  "No."

⚠️ BETTER BUT INCOMPLETE:
  "This could be more efficient."
  "I don't like this approach."

✅ EXCELLENT COMMENTS:
  "This loop calls the database on every iteration.
   Consider batching the IDs and doing a single
   SELECT ... WHERE id IN (ids) query. This would
   reduce from N queries to 1. Example:

   const users = await User.findAll({
     where: { id: userIds }
   });"

  "Optional: I noticed we have a similar function
   in utils/validation.js (line 45). Could we
   reuse that instead of duplicating? No blocker,
   just a thought."

  "Blocking: This function doesn't handle the case
   where the user isn't logged in (req.user could
   be undefined). We'd get a TypeError crash.
   Please add: if (!req.user) return res.status(401)..."

COMMENT TYPES TO USE:
  "Blocking: ..." → Must be fixed before merge
  "Optional: ..."  → Nice to have, but won't block merge
  "Question: ..."  → Seeking clarification, not demanding change
  "Nit: ..."       → Tiny style/preference note (very minor)
  "Praise: ..."    → Point out good things! (morale matters)
```

Responding to Review Feedback

```
# After receiving review comments:

# 1. Read all comments before making changes
# 2. For each comment, either:
#    - Make the change and reply "Done ✅"
#    - Disagree and explain why (respectfully!)
#    - Ask for clarification

# 3. Make your changes
git add changed-file.js
git commit -m "Address PR review: batch database queries"
git push origin feature/user-authentication

# The PR automatically updates with your new commits!
# Reviewers get notified and can re-review.

# GitHub tip: Use "Resolve conversation" button when
# a comment thread is fully addressed.
```

PR Best Practices

```
PULL REQUEST BEST PRACTICES
═══════════════════════════════════════════════════════════════

KEEP PRs SMALL:
  ✅ 200-400 lines changed = easy to review (30 min)
  ⚠️  400-800 lines changed = harder (1-2 hours)
  ❌ 800+ lines changed = very hard (reviewers give up)

  TIP: If your feature is large, split it into sequential PRs!
  PR 1: Add database models
  PR 2: Add API endpoints (builds on PR 1)
  PR 3: Add UI components (builds on PR 2)

ONE PURPOSE PER PR:
  ✅ "Add user authentication"
  ❌ "Add user auth + fix 10 unrelated bugs + refactor database"

  Mixed PRs make review hard and rollback impossible.

SELF-REVIEW FIRST:
  Before requesting review, open your own PR and read it.
  You'll catch ~50% of issues yourself!

RESPOND QUICKLY:
  If a reviewer comments, respond within 24 hours.
  Long delays stall everyone's work.

NEVER MERGE YOUR OWN PR:
  (Unless your team explicitly allows it for tiny fixes)
  The point is to have another set of eyes!
```

PR Automation (GitHub Actions Preview)

```yaml
# .github/workflows/pr-checks.yml
# This runs automatically on every PR!

name: PR Checks

on:
  pull_request:
    branches: [ main, develop ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Install dependencies
        run: npm install

      - name: Run tests
        run: npm test           # If tests fail, PR is blocked!

      - name: Check code style
        run: npm run lint       # If style fails, PR is blocked!

  # Optional: Auto-comment on PR with test results
  # Optional: Check for security vulnerabilities
  # Optional: Deploy to staging environment for testing
```

Assignment 3: Full Collaborative Workflow

Scenario

You're joining an existing development team for **"ShopEasy"** — an e-commerce platform. Your job is to implement a product search feature using the full professional Git workflow.

```
# === ASSIGNMENT 3: Complete Professional Git Workflow ===

# PART 1: SETUP (simulating joining a team)

# Fork the repository (on GitHub UI — click "Fork")
# Then clone YOUR fork:
git clone https://github.com/YOUR-USERNAME/shopeasy.git
cd shopeasy

# Add the original repo as upstream
git remote add upstream https://github.com/TEAM/shopeasy.git
git remote -v    # Verify both remotes

# Get the latest code
git fetch upstream
git switch main
git merge upstream/main

# PART 2: FEATURE DEVELOPMENT

# Create a descriptive feature branch
git switch develop
git pull upstream develop
git switch -c feature/SHOP-47-product-search

# Implement the feature incrementally (multiple commits)

# Commit 1: Create search infrastructure
mkdir -p src/search
cat > src/search/searchService.js << 'EOF'
/**
 * Product Search Service
 * Handles searching products by name, category, and price range
 */

class SearchService {
  constructor(productRepository) {
    this.productRepo = productRepository;
  }

  /**
   * Search products by query string
   * @param {string} query - Search terms
   * @param {Object} filters - Optional filters { category, minPrice, maxPrice }
   * @returns {Promise<Product[]>} Matching products
   */
```

```
    async search(query, filters = {}) {
      // Validate inputs
      if (!query || query.trim().length === 0) {
        throw new Error('Search query cannot be empty');
      }

      if (query.length > 100) {
        throw new Error('Search query too long (max 100 characters)');
      }

      // Build search parameters
      const searchParams = {
        query: query.trim().toLowerCase(),
        ...filters
      };

      // Validate price range
      if (searchParams.minPrice && searchParams.maxPrice) {
        if (searchParams.minPrice > searchParams.maxPrice) {
          throw new Error('Minimum price cannot exceed maximum price');
        }
      }

      return await this.productRepo.search(searchParams);
    }

    /**
     * Get search suggestions as user types
     * @param {string} partial - Partial search term
     * @returns {Promise<string[]>} Suggested search terms
     */
    async getSuggestions(partial) {
      if (!partial || partial.length < 2) {
        return [];
      }
      return await this.productRepo.getSearchSuggestions(partial.toLowerCase());
    }
  }

  module.exports = SearchService;
  EOF

  git add src/search/searchService.js
  git commit -m "Add SearchService with validation and suggestion support

  - Validate query length and emptiness
```

```
- Support optional category and price range filters
- Add getSuggestions() for autocomplete functionality
- Validates price range (min <= max)"

# Commit 2: Add tests
cat > tests/search/searchService.test.js << 'EOF'
const SearchService = require('../../src/search/searchService');

// Mock repository
const mockProductRepo = {
  search: jest.fn(),
  getSearchSuggestions: jest.fn()
};

describe('SearchService', () => {
  let searchService;

  beforeEach(() => {
    searchService = new SearchService(mockProductRepo);
    jest.clearAllMocks();
  });

  describe('search()', () => {
    test('successfully searches with valid query', async () => {
      const mockProducts = [{ id: 1, name: 'Red Shoes' }];
      mockProductRepo.search.mockResolvedValue(mockProducts);

      const results = await searchService.search('red shoes');

      expect(results).toEqual(mockProducts);
      expect(mockProductRepo.search).toHaveBeenCalledWith({
        query: 'red shoes',
      });
    });

    test('throws error for empty query', async () => {
      await expect(searchService.search('')).rejects.toThrow(
        'Search query cannot be empty'
      );
    });

    test('throws error for query over 100 characters', async () => {
      const longQuery = 'a'.repeat(101);
      await expect(searchService.search(longQuery)).rejects.toThrow(
        'Search query too long'
      );
```

```
    });

    test('applies filters when provided', async () => {
      mockProductRepo.search.mockResolvedValue([]);
      await searchService.search('shoes', {
        category: 'footwear',
        minPrice: 20,
        maxPrice: 100
      });

      expect(mockProductRepo.search).toHaveBeenCalledWith({
        query: 'shoes',
        category: 'footwear',
        minPrice: 20,
        maxPrice: 100
      });
    });

    test('throws error if min price exceeds max price', async () => {
      await expect(
        searchService.search('shoes', { minPrice: 100, maxPrice: 50 })
      ).rejects.toThrow('Minimum price cannot exceed maximum price');
    });
  });

  describe('getSuggestions()', () => {
    test('returns suggestions for 2+ character input', async () => {
      mockProductRepo.getSearchSuggestions.mockResolvedValue(['shoes', 'shorts']);

      const suggestions = await searchService.getSuggestions('sh');
      expect(suggestions).toEqual(['shoes', 'shorts']);
    });

    test('returns empty array for less than 2 characters', async () => {
      const suggestions = await searchService.getSuggestions('s');
      expect(suggestions).toEqual([]);
      expect(mockProductRepo.getSearchSuggestions).not.toHaveBeenCalled();
    });
  });
});
EOF

git add tests/search/searchService.test.js
git commit -m "Add comprehensive unit tests for SearchService

Tests cover:
```

```
- Successful search with and without filters
- Empty and oversized query validation
- Price range validation
- Autocomplete suggestion behavior"

# Commit 3: Add API endpoint
cat > src/search/searchController.js << 'EOF'
const SearchService = require('./searchService');
const { ProductRepository } = require('../repositories/productRepository');

const searchService = new SearchService(new ProductRepository());

/**
 * GET /api/search?q=shoes&category=footwear&minPrice=20&maxPrice=100
 */
async function searchProducts(req, res) {
  try {
    const { q, category, minPrice, maxPrice } = req.query;

    if (!q) {
      return res.status(400).json({ error: 'Query parameter "q" is required' });
    }

    const filters = {};
    if (category) filters.category = category;
    if (minPrice) filters.minPrice = parseFloat(minPrice);
    if (maxPrice) filters.maxPrice = parseFloat(maxPrice);

    const products = await searchService.search(q, filters);

    res.json({
      query: q,
      count: products.length,
      results: products
    });
  } catch (error) {
    if (error.message.includes('cannot be empty') ||
        error.message.includes('too long') ||
        error.message.includes('exceed maximum')) {
      return res.status(400).json({ error: error.message });
    }

    console.error('Search error:', error);
    res.status(500).json({ error: 'Search temporarily unavailable' });
  }
}
```

```
/**
 * GET /api/search/suggestions?q=sho
 */
async function getSearchSuggestions(req, res) {
  try {
    const { q } = req.query;
    const suggestions = await searchService.getSuggestions(q || '');
    res.json({ suggestions });
  } catch (error) {
    res.status(500).json({ error: 'Could not fetch suggestions' });
  }
}

module.exports = { searchProducts, getSearchSuggestions };
EOF

git add src/search/searchController.js
git commit -m "Add REST API endpoints for product search

Endpoints:
  GET /api/search?q=...        → Search with optional filters
  GET /api/search/suggestions  → Autocomplete suggestions

Features:
  - Proper HTTP status codes (400 for invalid input, 500 for server errors)
  - Filter parameters: category, minPrice, maxPrice
  - Error messages don't expose internal details to client"

# PART 3: PUSH AND OPEN PR

git push -u origin feature/SHOP-47-product-search

# Open PR on GitHub with detailed description:
# Title: "feat(search): Add product search with filters and autocomplete"
# Body: (Use the PR template shown earlier)
# Assignees: Your teammates
# Labels: "feature", "backend"
# Linked issue: #47

# PART 4: RESPOND TO REVIEW (simulated)

# Reviewer comment: "The parseFloat could produce NaN if non-numeric
# string is passed. Add validation."

# Make the fix
```

```
cat > src/search/searchController.js << 'EOF'
const SearchService = require('./searchService');
const { ProductRepository } = require('../repositories/productRepository');

const searchService = new SearchService(new ProductRepository());

async function searchProducts(req, res) {
  try {
    const { q, category, minPrice, maxPrice } = req.query;

    if (!q) {
      return res.status(400).json({ error: 'Query parameter "q" is required' });
    }

    const filters = {};
    if (category) filters.category = category;

    // FIXED: Validate numeric parameters
    if (minPrice !== undefined) {
      const min = parseFloat(minPrice);
      if (isNaN(min)) {
        return res.status(400).json({ error: 'minPrice must be a valid number' });
      }
      filters.minPrice = min;
    }

    if (maxPrice !== undefined) {
      const max = parseFloat(maxPrice);
      if (isNaN(max)) {
        return res.status(400).json({ error: 'maxPrice must be a valid number' });
      }
      filters.maxPrice = max;
    }

    const products = await searchService.search(q, filters);

    res.json({
      query: q,
      count: products.length,
      results: products
    });
  } catch (error) {
    if (error.message.includes('cannot be empty') ||
        error.message.includes('too long') ||
        error.message.includes('exceed maximum')) {
      return res.status(400).json({ error: error.message });
```

```
    }

    console.error('Search error:', error);
    res.status(500).json({ error: 'Search temporarily unavailable' });
  }
}

async function getSearchSuggestions(req, res) {
  try {
    const { q } = req.query;
    const suggestions = await searchService.getSuggestions(q || '');
    res.json({ suggestions });
  } catch (error) {
    res.status(500).json({ error: 'Could not fetch suggestions' });
  }
}

module.exports = { searchProducts, getSearchSuggestions };
EOF

git add src/search/searchController.js
git commit -m "Fix: Validate numeric query params to prevent NaN values

Address review feedback: parseFloat('abc') returns NaN.
Now explicitly validate minPrice and maxPrice are numbers
and return 400 Bad Request if they're not."

git push origin feature/SHOP-47-product-search

# The PR automatically updates — reviewer sees new commits!
# Reviewer approves → Merge!

# PART 5: CLEANUP AFTER MERGE

git switch develop
git pull upstream develop    # Get the merged changes

git branch -d feature/SHOP-47-product-search             # Local cleanup
git push origin --delete feature/SHOP-47-product-search   # Remote cleanup

echo "Feature complete! 🎉"

# PART 6: VIEW FINAL HISTORY
git log --oneline --graph --decorate
```

Assessment Rubric for Assignment 3

Criteria	Excellent	Good	Needs Work
Branch Naming	Follows convention (type/TICKET-desc)	Descriptive	Unclear/generic
Commit Granularity	Each commit is one logical change	Mostly logical	Too large or too small
Commit Messages	Clear, imperative, explains why	Descriptive	Vague
PR Description	Complete, linked to issue, has checklist	Mostly complete	Missing key info
Code Quality	Clean, tested, documented	Mostly clean	Issues present
Review Response	Responds quickly, professional	Responds	Ignores feedback
Cleanup	Branch deleted, remote cleaned	Local cleaned	Left branches around

Final Quick Reference: The Complete Git Cheat Sheet

```
═══════════════════════════════════════════════════════════════
                    GIT COMPLETE CHEAT SHEET
═══════════════════════════════════════════════════════════════

SETUP
  git config --global user.name "Name"     Set your name
  git config --global user.email "email"   Set your email
  git config --global alias.lg "log --oneline --graph --decorate"

STARTING A PROJECT
  git init                      Create new local repository
  git clone <url>               Clone a remote repository

DAILY WORKFLOW
  git status                    Check status of working directory
  git add <file>                Stage a file
  git add .                     Stage all changes
  git add -p                    Stage changes interactively
  git commit -m "msg"           Commit with message
  git commit -am "msg"          Stage + commit (tracked files only)

HISTORY
  git log                       Full history
  git log --oneline             Compact history
  git log --graph --oneline     Visual branch history
  git log --author="Name"       Filter by author
  git show <hash>               Show a specific commit's changes
  git diff                      Unstaged changes
  git diff --staged             Staged changes

UNDOING
  git restore <file>            Discard working directory changes
  git restore --staged <file>   Unstage a file
  git revert <hash>             Safely undo a commit (adds new commit)
  git reset --soft HEAD~1       Undo commit, keep staged
  git reset HEAD~1              Undo commit, keep in working dir
  git reset --hard HEAD~1       Undo commit, discard changes ⚠️

BRANCHING
  git branch                    List local branches
  git branch -a                 List all branches
  git switch <branch>           Switch to a branch
  git switch -c <branch>        Create and switch
```

```
  git branch -d <branch>       Delete merged branch
  git branch -D <branch>       Force delete

MERGING & REBASING
  git merge <branch>           Merge a branch
  git merge --no-ff <branch>   Merge with merge commit always
  git merge --squash <branch>  Squash all commits into one
  git merge --abort            Cancel merge in progress
  git rebase <branch>          Rebase current branch onto another
  git rebase -i HEAD~n         Interactive rebase (last n commits)
  git rebase --continue        Continue after resolving conflict
  git rebase --abort           Cancel rebase in progress

REMOTE
  git remote -v                List remotes
  git remote add origin <url>  Add a remote
  git fetch                    Download changes (don't merge)
  git pull                     Fetch + merge
  git pull --rebase            Fetch + rebase (cleaner)
  git push                     Push current branch
  git push -u origin <branch>  Push and set upstream
  git push --force-with-lease  Force push (safer than --force)
  git push origin --delete <b> Delete remote branch

STASHING (save work temporarily)
  git stash                    Stash current changes
  git stash pop                Restore stashed changes
  git stash list               List all stashes

TAGS (marking releases)
  git tag v1.0                 Create a lightweight tag
  git tag -a v1.0 -m "msg"    Create annotated tag
  git push --tags              Push tags to remote

═══════════════════════════════════════════════════════════════
```

Program Wrap-Up

What You've Accomplished in 3 Days

DAY 1 MASTERED:
- ✅ Git concepts
- ✅ Installation/config
- ✅ Repository creation
- ✅ Staging & committing
- ✅ History viewing
- ✅ Undoing changes
- ✅ Basic branching
- ✅ Basic merging

DAY 2 MASTERED:
- ✅ Advanced branching
- ✅ Git Flow strategy
- ✅ Feature branches
- ✅ Hotfix branches
- ✅ Complex merging
- ✅ Conflict resolution
- ✅ Rebase vs Merge
- ✅ Interactive rebase

DAY 3 MASTERED:
- ✅ Remote workflows
- ✅ Forking & cloning
- ✅ Push & pull
- ✅ Pull Requests
- ✅ Code review
- ✅ Team workflows
- ✅ Professional habits

The Developer's Git Philosophy

```
COMMIT EARLY, COMMIT OFTEN
→ Small commits are easier to understand and revert

BRANCHES ARE FREE — USE THEM
→ Never work directly on main

THE COMMIT MESSAGE IS A GIFT TO YOUR FUTURE SELF
→ Write it as if you'll be debugging at 3 AM

PULL BEFORE YOU PUSH
→ Always get the latest before sharing your work

CODE REVIEW IS A COLLABORATION, NOT A JUDGMENT
→ Give the feedback you'd want to receive
```

Additional Resources

Resource	URL	Best For
Pro Git Book (free)	git-scm.com/book	Complete reference
Learn Git Branching	learngitbranching.js.org	Interactive visual learning

Resource	URL	Best For
GitHub Skills	skills.github.com	Hands-on GitHub courses
Atlassian Git Tutorials	atlassian.com/git	Excellent explanations
Oh Shit, Git!	ohshitgit.com	Fixing common disasters
Git Visualizer	git-school.github.io/visualizing-git	See commands visually
Conventional Commits	conventionalcommits.org	Commit message standard

End of Source Code Management with Git — 3-Day Intensive Training Guide

Created for beginners. Designed for mastery. Built for the real world.